



微算機系統實習 MICROPROCESSOR SYSTEMS LAB. AUTUMN, 2026

Instructor : Yen-Lin Chen(陳彥霖), Ph.D. Professor
Dept. Computer Science and Information Engineering
National Taipei University of Technology



LECTURE 1 - 嵌入式系統開發與實驗平台簡介

Environment Introduction



本課程作業及報告上傳

- 本課程課後討論FB社群：www.facebook.com/groups/ntut.115-1.tx2/
- 位址：北科i學園plus
- 請各位同學先將分組的名單填入FB社團中的表單，接著我們會將你們分組，你們可以查看你們各自的組別。
- 作業上傳格式：以組別命名，壓縮為zip，例如：組別X.zip
- 其zip裡要包含以下資料夾
 - 程式碼：存放程式碼
 - 報告：存放報告 (PDF檔)
- 請依照此規則上傳，否則會造成助教改作業的困擾，成績有問題要自行負責
- 若有其他有關此作業的備註訊息要跟TA告知，可在根目錄附上文字檔 (Readme.txt)，並留下你的E-Mail

嵌入式處理器簡介



前言

- ARM以低功耗、高效能聞名，廣泛應用於行動裝置、單晶片電腦、物聯網設備與邊緣運算平台
- NVIDIA Jetson TX2 採用 ARM Cortex-A57 多核心處理器與 Pascal GPU，內建支援 Ubuntu Linux 作業系統
- 學習在 ARM 平台上開發嵌入式系統程式，並結合 Linux 作業系統，是未來 AIoT 與智慧裝置領域不可或缺的技能。



ARM架構

- 在處理核心當中，以ARM架構最為熱門，然而，ARM架構與一般x86有什麼不一樣？
- 其最主要的差異在於指令集，Reduced Instruction Set Computing(RISC)以及Complex Instruction Set Computing(CISC)。
- TX2 採用 ARM Cortex-A57 多核心處理器與 Pascal GPU，內建支援 Ubuntu Linux 作業系統
- 學習在 ARM 平台上開發嵌入式系統程式，並結合 Linux 作業系統，是未來 AIoT 與智慧裝置領域不可或缺的技能。

※指令集：用來命令處理核心該進行哪些處理或是運算所用，而指令集的差異在於，一道指令可以完成的事情。



ARM優點

- 高效能：

由於 ARM 架構的指令集比較簡單，可以在較短的時間內完成更多的指令，因此 ARM 架構的處理器速度相對較快。

- 低功耗：

ARM 架構通常具有低功耗特性，這使得它們在移動設備和嵌入式系統中非常受歡迎。ARM 架構支援多種功耗管理技術，例如，動態電壓調節 (Dynamic Voltage Scaling, DVS)、裝載/卸載資源管理等，這些技術可以最大程度地節省系統功耗。



ARM優點

- 體積小：

ARM架構的設計簡單、結構緊湊，因此它們非常適合用於體積受限的設備中，例如：智能手機、平板電腦、可攜式電子產品等。此外，由於ARM處理器設計簡單，它們的封裝體積也相對較小，這進一步減小了整個系統的體積。

- 可擴展性：

ARM架構可以用於從微控制器到高性能伺服器等廣泛的應用場景，而且ARM處理器的設計可以根據需求進行擴展。例如，ARM處理器可以擴展到多核設計，以實現更高的性能。



ARM優點

- 易於整合：

ARM架構通常可以與其他硬體和軟體組件輕鬆整合，例如，許多智能手機和平板電腦中都使用ARM處理器和Android操作系統。

- 節省成本：

由於ARM處理器設計簡單、功耗低、性能高，因此它們通常比其他處理器更便宜。此外，由於ARM架構應用非常廣泛，因此開發和維護ARM平台的成本也相對較低。



指令集架構

- 接著，如何區分這些指令集？一個依據為，「暫存器」的使用。因為資料要透過CPU進行運算，需要先存放在暫存器當中，接著讓CPU讀取、運算，而兩個指令集架構最大的差異在那？
 - CISC：一道指令可以執行多種運算處理。
 - RISC：一道指令，執行一種運算處理。
- 那一種指令集比較花費「暫存器」？RISC，為什麼？因為RISC在設計初期，有pipeline的機制讓CPU隨時保持在Busy的狀態，不會等待指令進來。但是pipeline需要事先把指令以及資料都存放在暫存器當中，所以造就了RISC需要較多的暫存器存放資料，但是效能將會較好。



智慧終端的抉擇

- 為什麼現今智慧終端普遍使用RISC架構，也就是ARM架構的解決方案？「省電」，為什麼RISC架構會比CISC架構省電？因為「硬體架構」，由於CISC的設計是透過增加硬體的設計來讓CPU可以同時處理多種運算，造就了需要更多的電力來驅動CPU，然而RISC的硬體架構較為簡單，所以需要的電力，相對較低，這也是為什麼RISC普遍被用於行動裝置的主要原因。



ARM架構的演進

- 在ARM架構的演進主要分為：
 - Von Neumann Architecture
 - Harvard Architecture
- 這兩個架構當中，主要的區別在記憶體架構，
 - 集中存放
 - Data與Instructions分別存放。
- 效能考量，Harvard Architecture可以進行指令預讀取，進而提升執行效率。

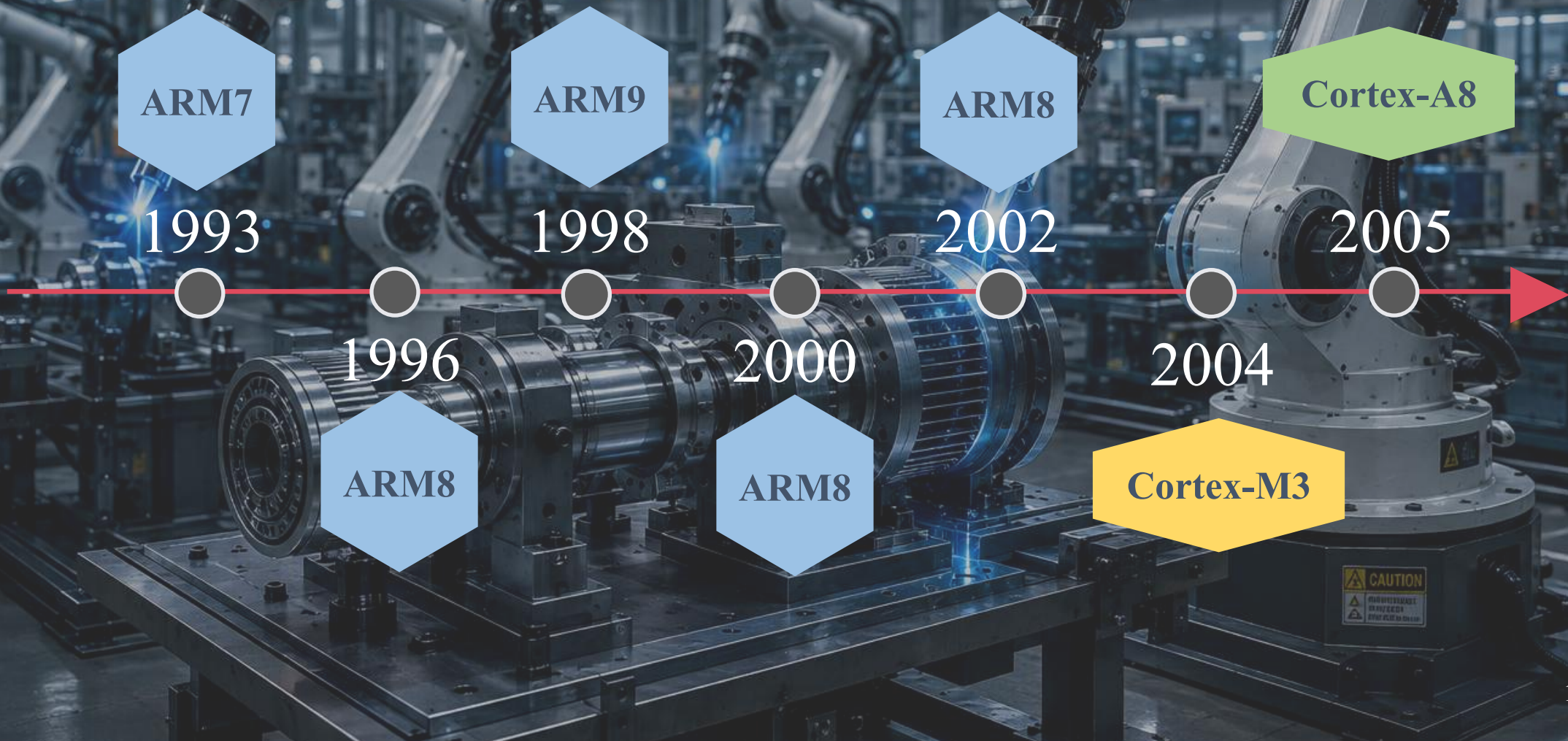


ARM經營模式

- ARM，現階段為一家提供「CPU」設計架構的公司，提供架構設計作為參考設計，並授權給其他CPU生產公司進行參考、修改。
例如：
 - Qualcomm
 - Texas Instruments
 - NVIDIA
- 讓各個公司可以根據這些基礎架構下，建立符合自家產品的定位。
例如：
 - Qualcomm：整合Baseband processor
 - Texas Instruments：整合DSP
 - NVIDIA：整合繪圖晶片

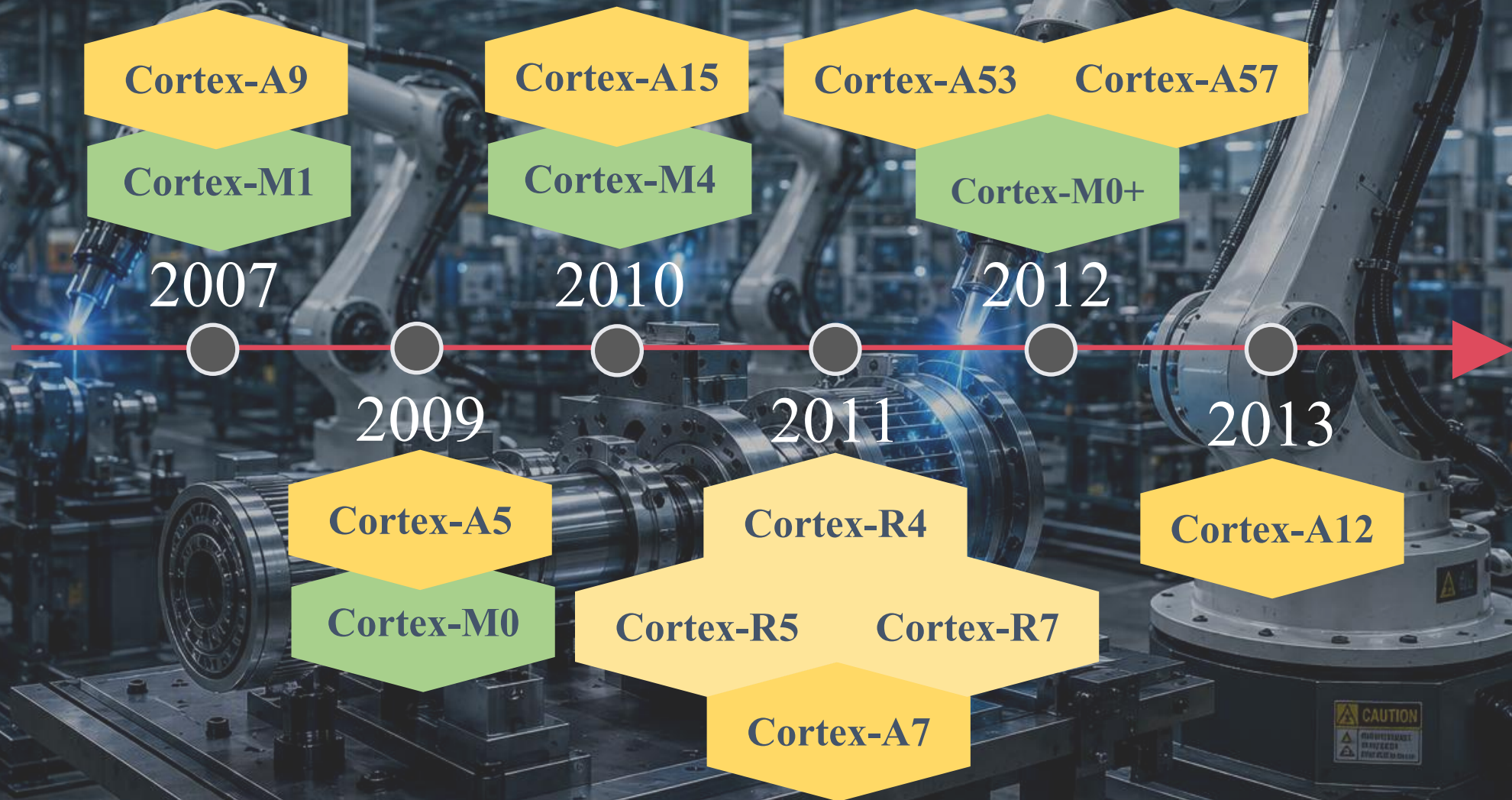


ARM 架構演進



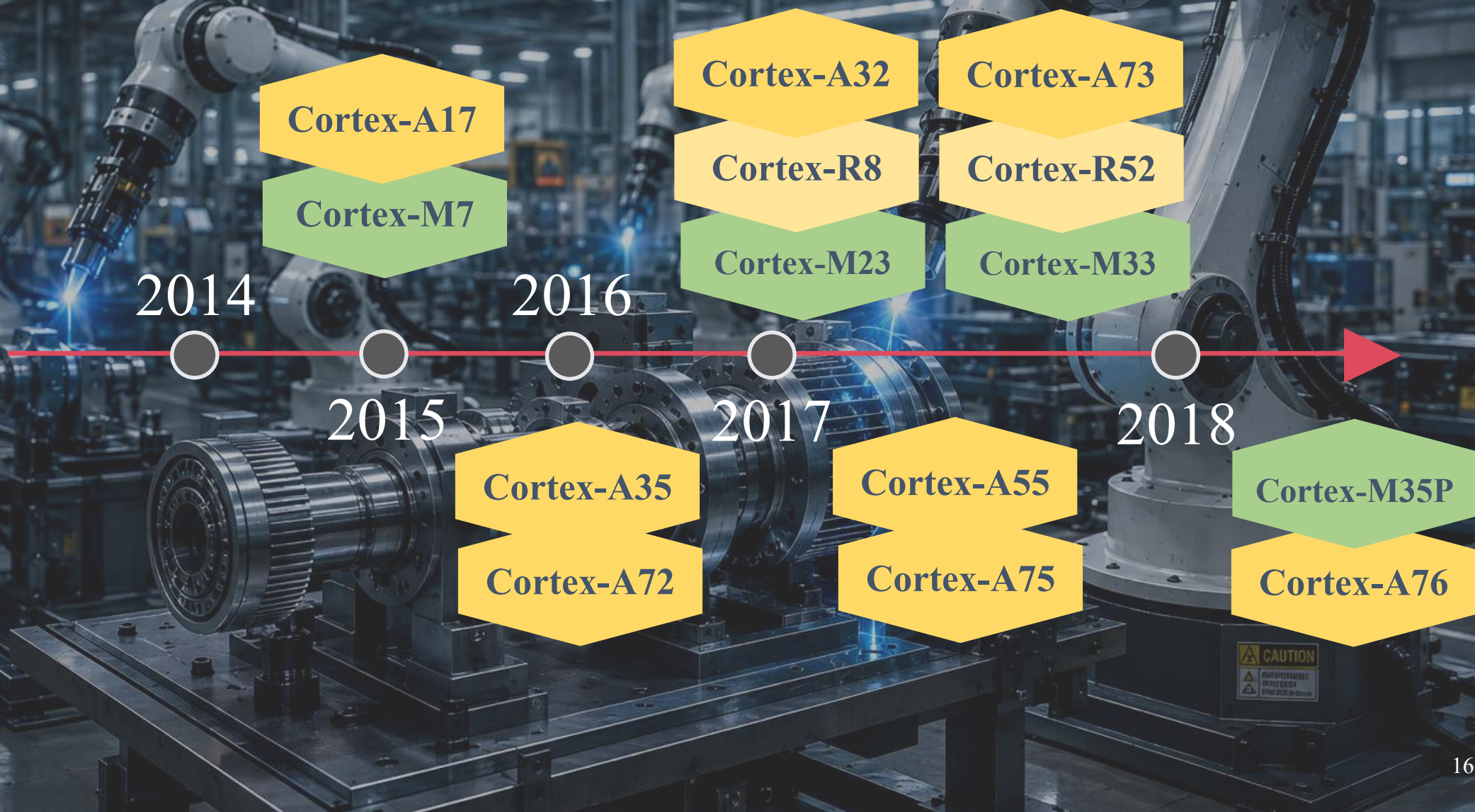


ARM 架構演進





ARM 架構演進





ARM 架構演進

Cortex-A77

2019

Cortex-A77應用：

- 高通 Snapdragon 865
- 聯發科 天璣 1000
- 小米10

Cortex-A510

Cortex-A710

Cortex-X2

2021

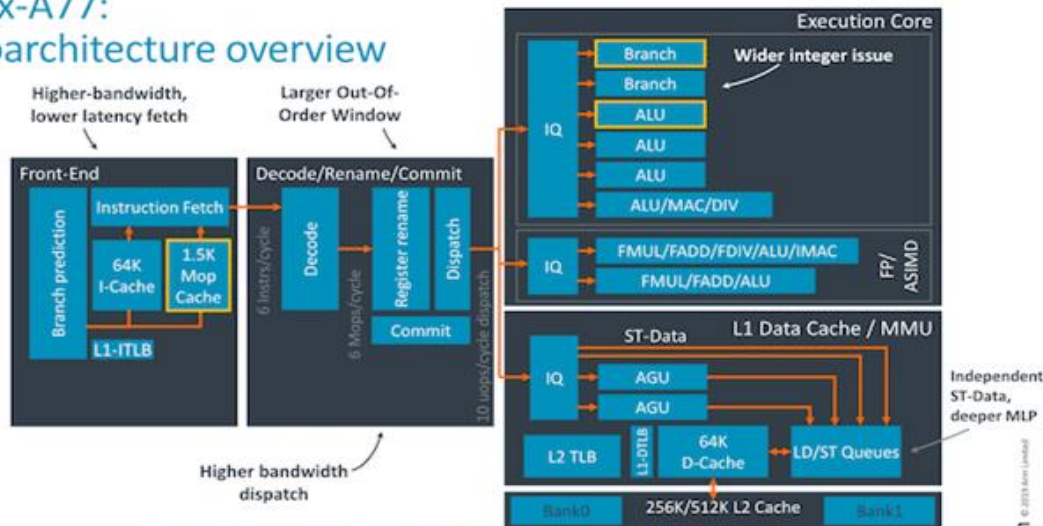
2020

Cortex-M55

Cortex-A78

Cortex-X1

Cortex-A77: Microarchitecture overview



The embargo for this content presented at Arm Tech Day will lift on Sunday, May 26 at 9:00 p.m. PT. Corresponding UK and Taiwan times are: Monday, May 27 at 5:00 a.m. BST / Monday, May 27 at 12:00 p.m. China Standard Time



ARM 架構演進

Cortex-A715 & X3應用：

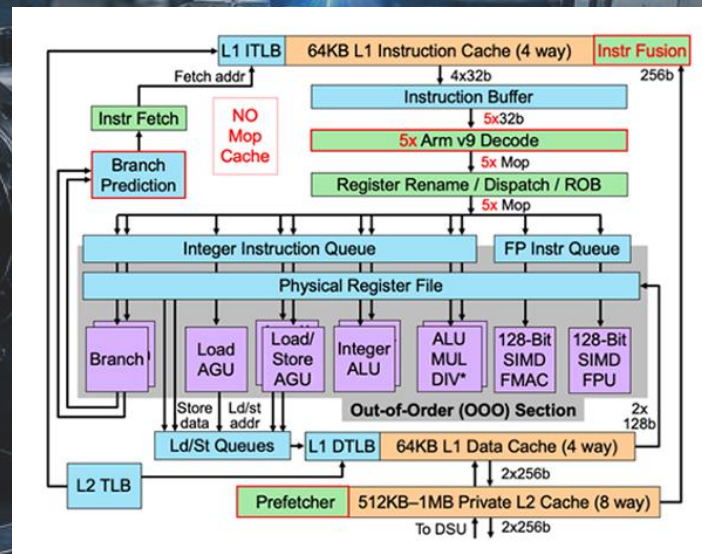
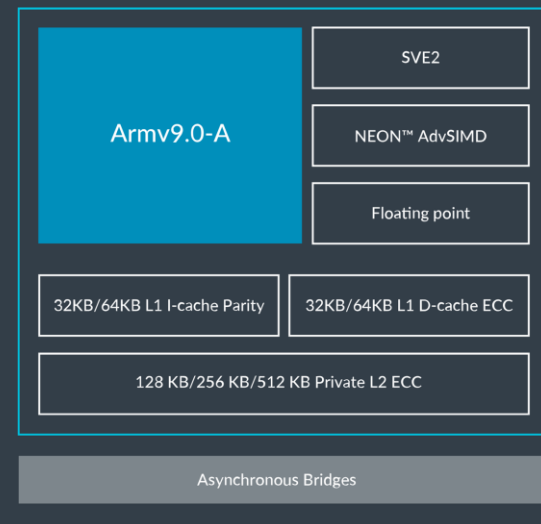
- 高通 驍龍8(+) Gen 2
- 聯發科 天璣 9200

2022

Cortex-A715

Cortex-X3

arm
CORTEX®-A715



Cortex-A715 microarchitecture



ARM 架構演進

Cortex-A720 & A520 & X4特色：

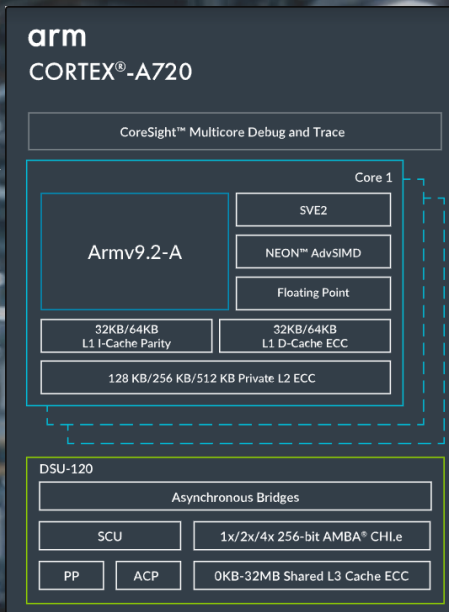
- 搭載ARMV9.2的CPU
- 不再支援32bit
- 性能、效率提升：
 - Cortex-X4性能提升15%
 - Cortex-A720效率提升20%
 - Cortex-A520效率提升22%

2023

Cortex-A720

Cortex-A520

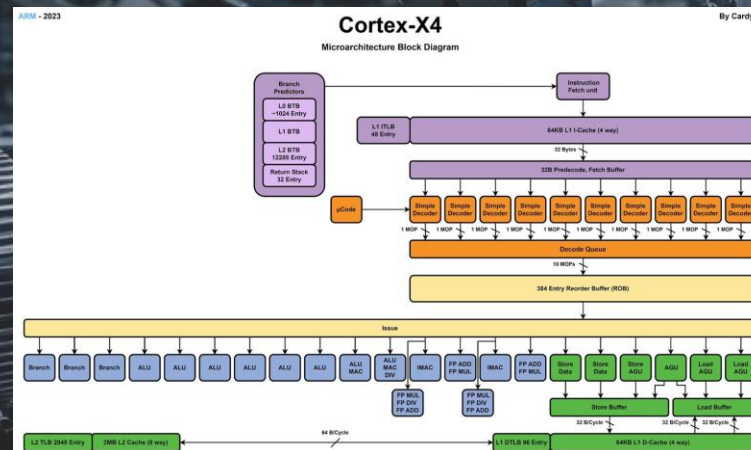
Cortex-X4



2024

Arm's New Lumex CSS Platform

- Lumex 將最高效能的 CPU 與可擴展矩陣擴展版本 2 (SME2)、GPU 和系統 IP 結合在一起
- 平台提供一致的 AI 性能、簡化的開發者支援以及廣泛的設備適用性，涵蓋從高階智慧型手機到成本受限的細分市場



Cortex-X4 microarchitecture





ARM 架構演進

Arm's New Lumex CSS Platform

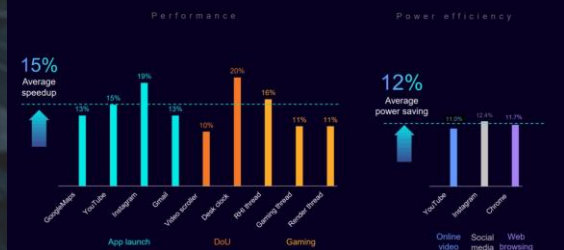
- Lumex 將最高效能的 CPU 與可擴展矩陣擴展版本 2 (SME2)、GPU 和系統 IP 結合在一起

2025

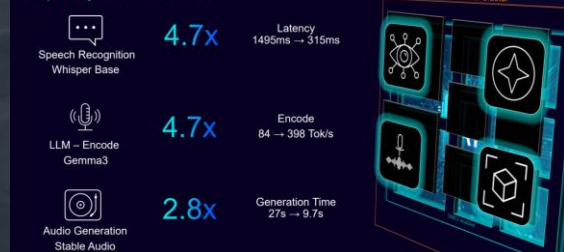
Lumex CSS 平台的核心功能

- ✓ 基於 Armv9.3-A 和 Arm SME2 構建，用於專用 AI 計算
- ✓ 包括帶有光線追蹤 v2 的 Arm Mali GPU，可實現 AI 增強視覺效果
- ✓ 透過以 AI 所需的速度和規模實現創新，縮短產品上市時間
- ✓ 透過統一的軟體和開發人員工具實現 AI 準備

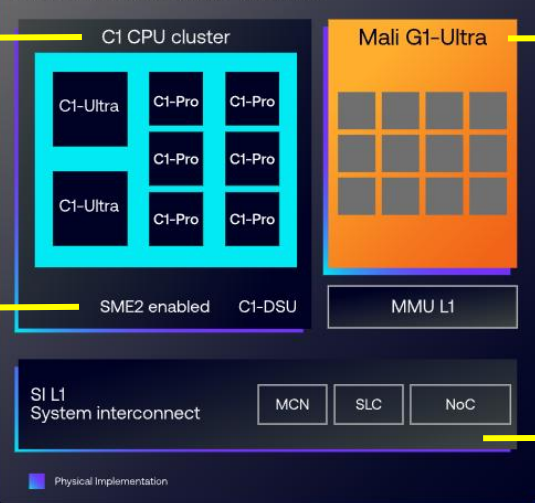
C1 CPU Cluster: Elevating User Experience in Real Workload Applications



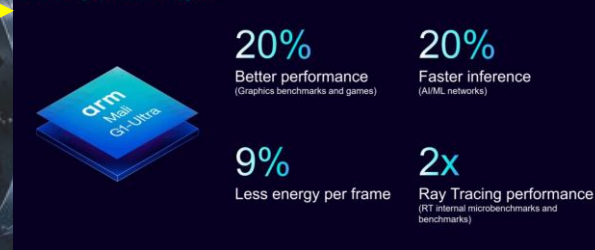
SME2: Accelerating the Latest GenAI Workloads Transparently enabled via KleidAI



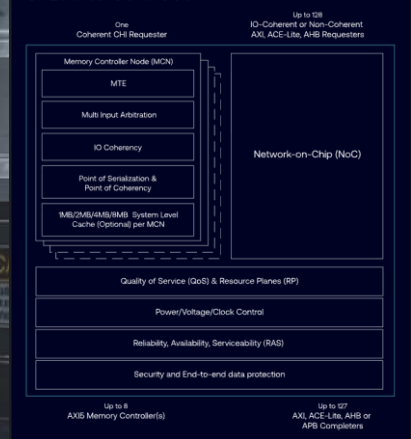
Arm Lumex CSS Platform



Mali G1-Ultra: The Pinnacle of Mobile GPU Performance and Efficiency Double-digit performance gains



arm SI L1 Interconnect





ARM 架構演進

2026 ARM 最新科技：端側生成式 AI 爆發

2026

微架構全面升級：

旗艦級 Cortex-X925 核心（代號 Blackhawk）與高效能 Cortex-A725 核心成為市場主流。

全面採用頂級 3nm / 2nm 頂尖製程，大幅降低極限算力下的系統功耗。

次世代指令集與硬體加速：

普及 Armv9.4-A / Armv9.5-A 指令集架構。

硬體層面深度優化可擴展矩陣擴展（SME2）與高 TOPS 運算能力。

專為邊緣端（Edge）設備直接執行大語言模型（LLM，如 Llama 3、Gemma）提供硬體級加速。

應用實例：

聯發科：天璣 9400 / 9500 系列，採用「全大核」架構，引領端側生成式 AI 手機普及。

物聯網與微控制器：Cortex-M85 成為次世代智慧硬體明星，讓極低功耗單晶片（MCU）不需外接 GPU 即可在地端執行生成式語音與影像辨識。

Future



ARM處理器使用

- 消費性電子上如：
 - 手機
 - 掌上型遊戲機
 - 智慧終端
 - 數位電視
 - 機上盒
 - 電腦周邊產品



Apple在WWDC 2020宣布邁向ARM架構，並於2025宣佈macOS 26 Tahoe為最後支援Intel的版本，正式邁向Apple Silicon架構

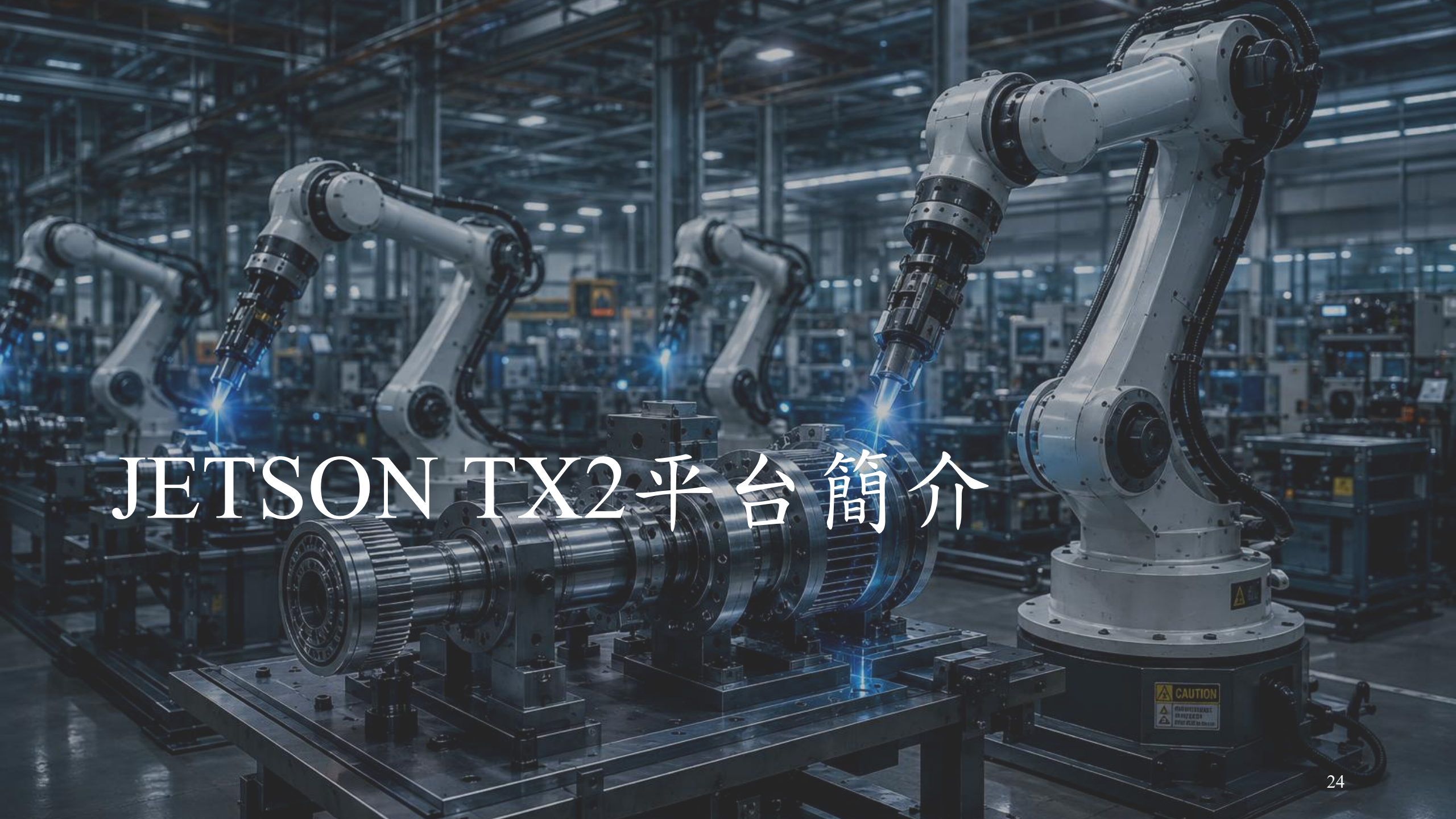


任天堂二次採用NVIDIA的晶片，並採用ARM架構晶片



ARM架構定位

- Cortex-A系列：Application應用，例如：手機
- Cortex-M系列：Microcontroller應用，常應用於工業控制的用途。
- Cortex-R系列：Real-time應用，例如：硬碟控制、引擎管理、Baseband
- 即時處理器核心...等。
- Cortex-X系列：高性能的移動設備和嵌入式系統，例如：智能手機、平板電腦、智能穿戴設備...等。

The background image shows a factory floor with several white industrial robotic arms. They are positioned around a large, complex metal assembly that is being welded. Bright blue sparks are visible at the points where the welding torches meet the metal. The scene is dimly lit, with the primary light source being the sparks from the welding process. The overall tone is industrial and technological.

JETSON TX2平台簡介



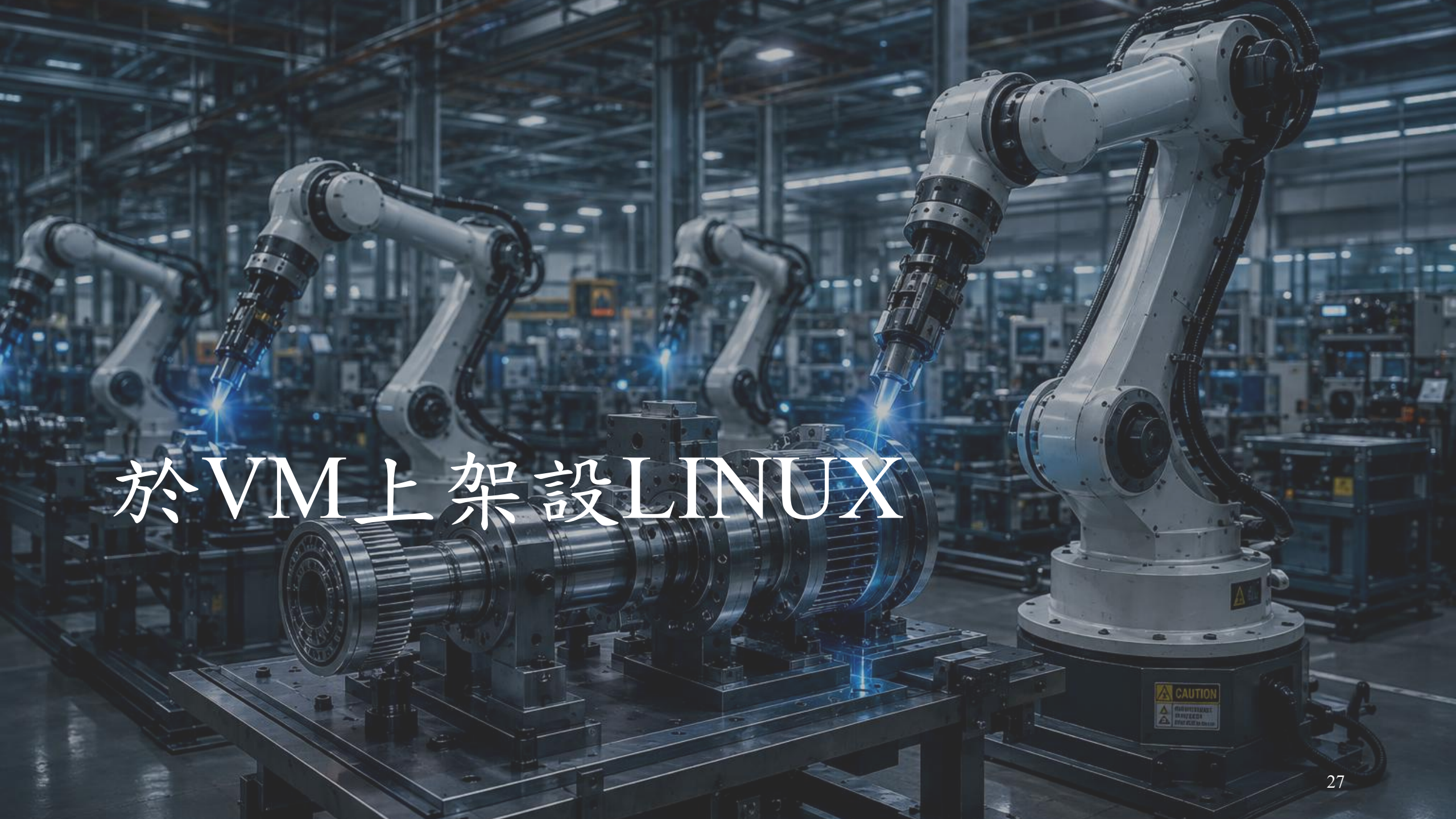
NVIDIA JETSON TX2

- 雙核心 Denver 2 64-bit CPU + 四核心 ARM® A57 Complex
- 8 GB L128 bit DDR4 記憶體
- 32 GB eMMC 5.1 Flash 儲存
- 可連接支持802.11ac WLAN 和 藍芽的裝置
- 10/100/1000BASE-T 乙太網路
- USB 3.0 Type A
- USB 2.0 Micro-AB (支持recovery 與 host 模式)
- HDMI
- M.2 Key E
- PCI-E x4
- Gigabit Ethernet
- Full-Size SD
- SATA Data and Power
- GPIOs, I2C, I2S, SPI, CAN*
- TTL UART with Flow Control
- Display Expansion Header
- Camera Expansion Header



NVIDIA Jetson Xavier NX

- GPU 21 兆次運算 384-core NVIDIA Volta™ GPU with 48 Tensor Cores
- CPU 6 核心 NVIDIA Carmel ARM®v8.2 64 位元 CPU , 6 MB L2 + 4 MB L3
- 2 X NVDLA (NVIDIA Deep Learning Accelerator)
- 記憶體 8 GB 128 位元 LPDDR4x , 51.2GB/秒
- 儲存空間 16 GB eMMC 5.1
- 電源 10W/15W
- USB 3.1 TYPE A
- USB 2.0 MICRO B
- HDMI
- DisplayPort
- M.2 Key M
- M.2 Key E
- Gigabit Ethernet
- Connects to 802.11ac WLAN and Bluetooth 5.0 enable devices
- 40-pin expansion headers(UART, SPI, I2S, I2C, PWM, GPIO)
- 2 x MIPI CSI-2 camera connector

The background image shows a dimly lit industrial factory floor. Several white robotic arms are visible, some of which are emitting a bright blue light from their nozzles. In the foreground, a large, complex mechanical assembly, possibly a motor or a large gear, is mounted on a metal frame. The overall atmosphere is futuristic and technological.

於VM上架設Linux



於VMWARE上安裝UBUNTU 18.04 LTS

- 為了能夠在Windows下同時使用Linux及Windows，故必須利用VMWare建立Virtual Machine，並在Virtual Machine上安裝Linux，以利往後建立嵌入式系統開發環境。
- 介紹如何透過VMWare建立Virtual Machine及安裝Linux (Ubuntu)



於VMWARE上安裝UBUNTU 18.04 LTS

- 安裝Ubuntu 18.04 LTS 64-bit 於 VMWare
 - VMware帳號註冊說明
 - 網址：
 - <https://knowledge.broadcom.com/external/article/145581/register-for-an-account-on-the-broadcom.html>
 - 下載VMWare Workstation安裝說明
 - 網址：<https://knowledge.broadcom.com/external/article/368734/download-desktop-hypervisor-workstation.html>
- 下載Ubuntu 18.04 LTS 64-bit並安裝在VMWare上
 - 網址：<http://releases.ubuntu.com/18.04/>
 - 下載 `ubuntu-18.04.5-desktop-amd64.iso`



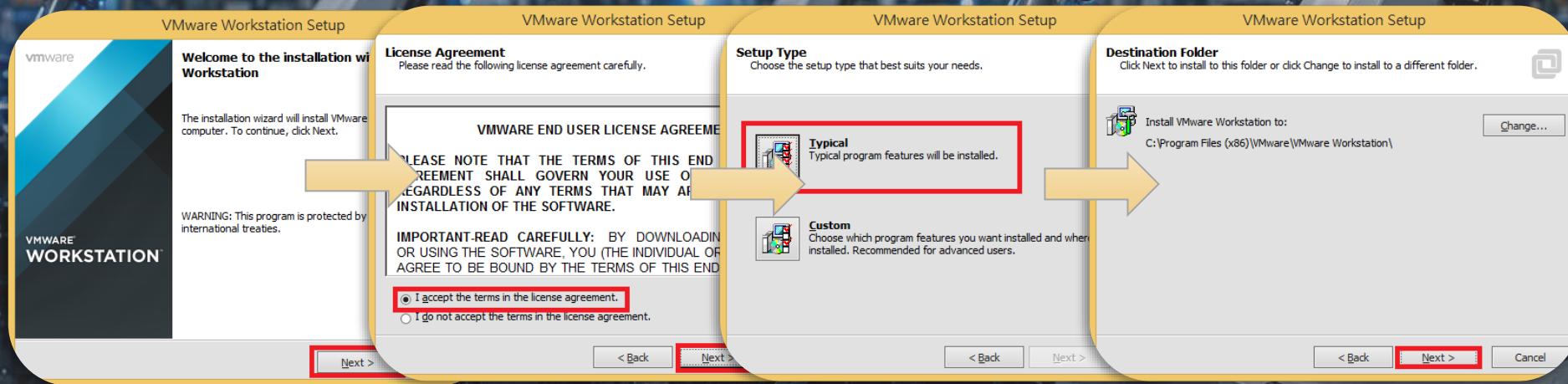
於VMWARE上安裝UBUNTU 18.04 LTS

Name	Last modified	Size	Description
 Parent Directory		-	
 MD5SUMS-metalink	2020-02-12 13:42	296	
 MD5SUMS-metalink.gpg	2020-02-12 13:42	916	
 SHA256SUMS	2020-08-13 15:39	202	
 SHA256SUMS.gpg	2020-08-17 12:28	833	
 ubuntu-18.04.5-desktop-amd64.iso	2020-08-06 22:59	2.0G	Desktop image for 64-bit PC (AMD64) computers (standard download)
 ubuntu-18.04.5-desktop-amd64.iso.torrent	2020-08-13 15:02	164K	Desktop image for 64-bit PC (AMD64) computers (BitTorrent download)
 ubuntu-18.04.5-desktop-amd64.iso.zsync	2020-08-13 15:02	4.1M	Desktop image for 64-bit PC (AMD64) computers (zsync metafile)



環境建立-安裝VMWARE

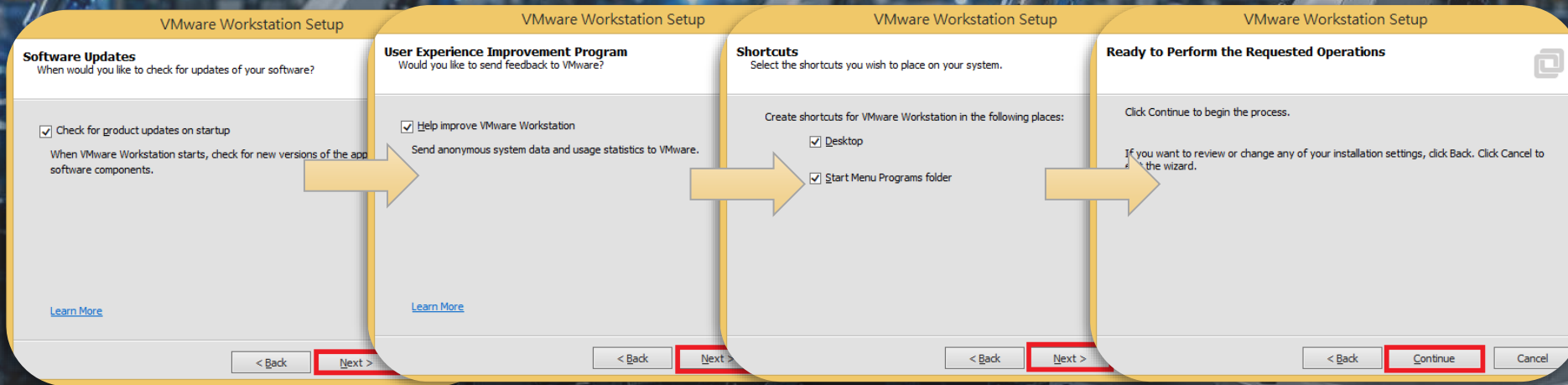
- 安裝 VMWare
- 基本上，建議都照預設值安裝





環境建立-安裝VMWARE

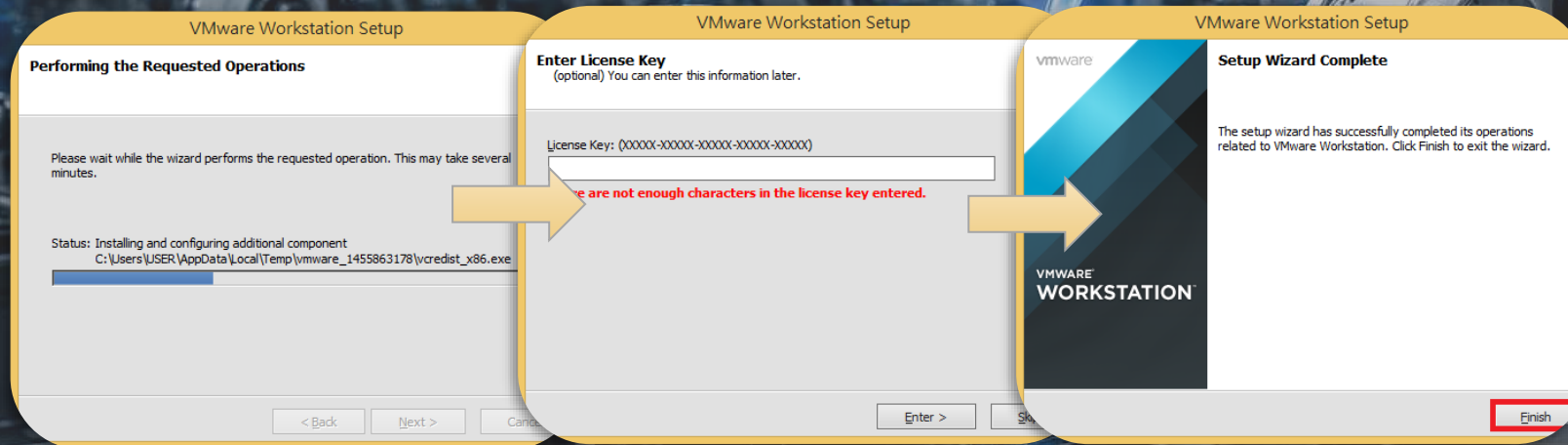
- 安裝 VMWare
- 基本上，建議都照預設值安裝





環境建立-安裝VMWARE

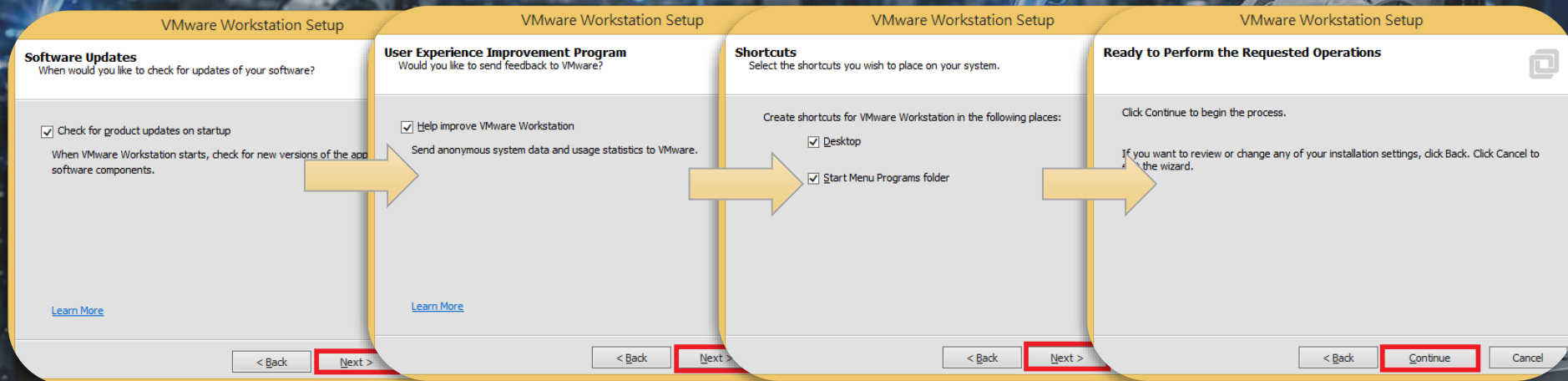
- 安裝 VMWare
- 基本上，建議都照預設值安裝





環境建立-安裝VMWARE

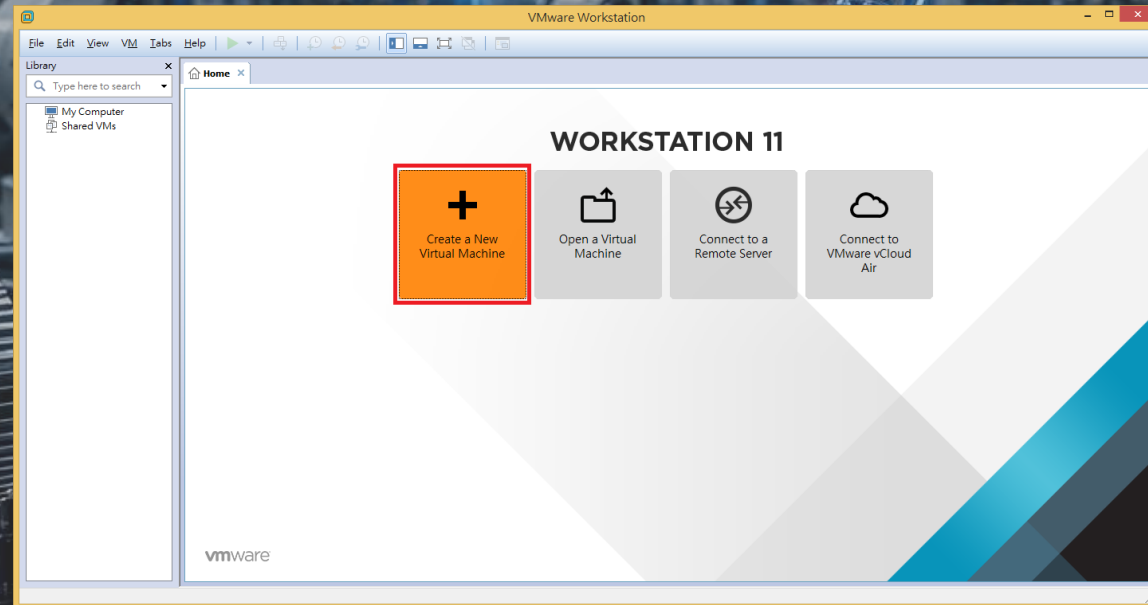
- 安裝 VMWare
- 基本上，建議都照預設值安裝





環境建立-安裝UBUNTU 18.04 LTS

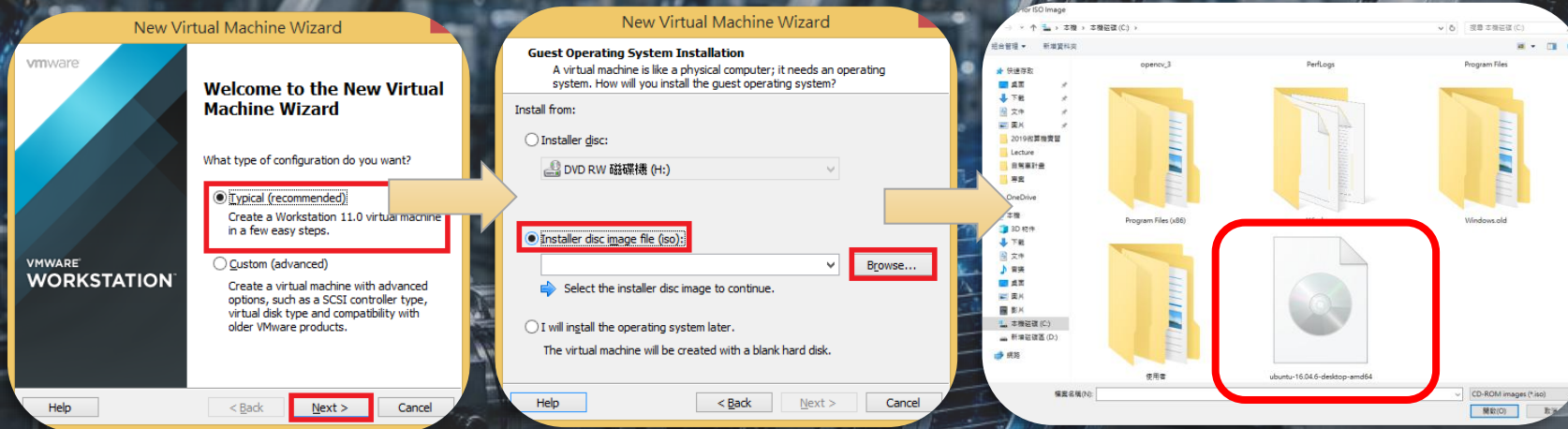
- 新增模擬機器
 - 作業系統：Linux
 - 版本：Ubuntu-64bit





環境建立-安裝UBUNTU 18.04 LTS

- 載入剛下載的Ubuntu 18.04 iso檔





環境建立-安裝UBUNTU 18.04 LTS

- 輸入資訊
 - User Name: nvidia
 - Password: nvidia

New Virtual Machine Wizard

Welcome to the New Virtual Machine Wizard
A virtual machine is like a physical computer; it needs an operating system. How will you install the guest operating system?

Install from:

☐ Installer disc:
DVD RW 磁碟機 (H:)

☒ Installer disc image file (iso):
C:\ubuntu-16.04.6-desktop-amd64.iso Browse...

☐ Ubuntu 64-bit 16.04.6 detected.
This operating system will use Easy Install. [\(What's this?\)](#)

☐ I will install the operating system later.
The virtual machine will be created with a blank hard disk.

Help < Back **Next >** Cancel

New Virtual Machine Wizard

Easy Install Information
This is used to install Ubuntu 64-bit.

Personalize Linux

Full name: TX2

User name: nvidia

Password:
Confirm:

Help < Back **Next >** Cancel

New Virtual Machine Wizard

Name the Virtual Machine
What name would you like to use for this virtual machine?

Virtual machine name:
TX2

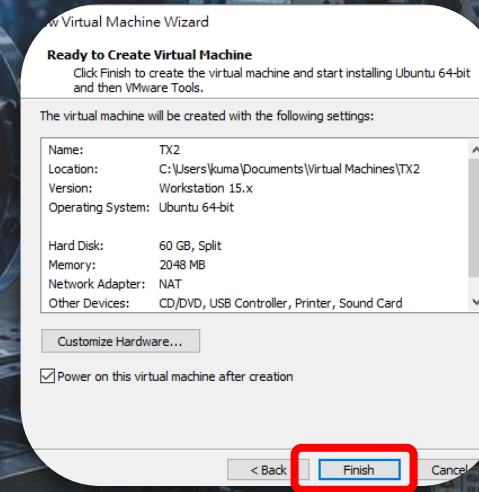
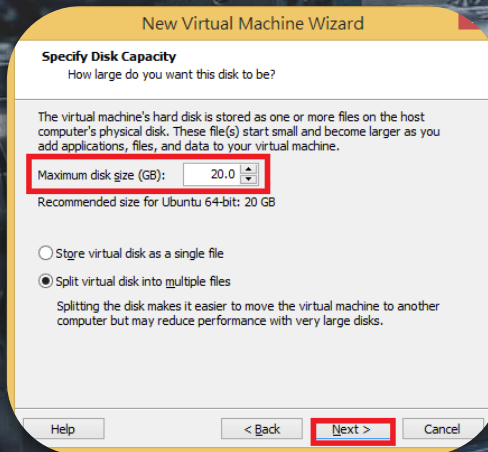
Location:
C:\Users\kuma\Documents\Virtual Machines\TX2 Browse...

< Back **Next >** Cancel



環境建立-安裝UBUNTU 18.04 LTS

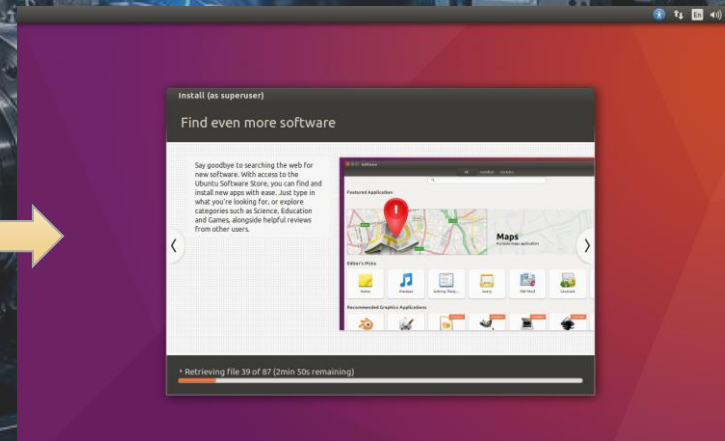
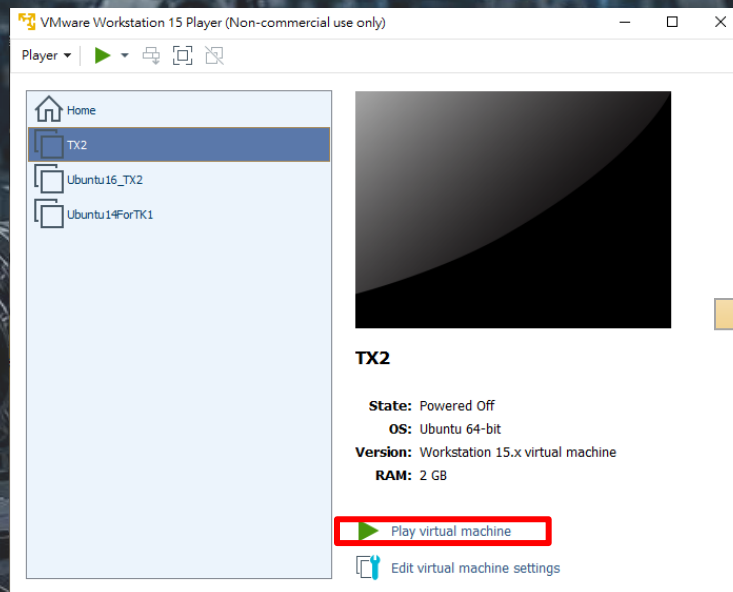
- 選擇disk size
 - 推薦100GB，如硬碟容量夠的話，可以自行增加
- 選擇RAM size
 - 推薦大於8GB

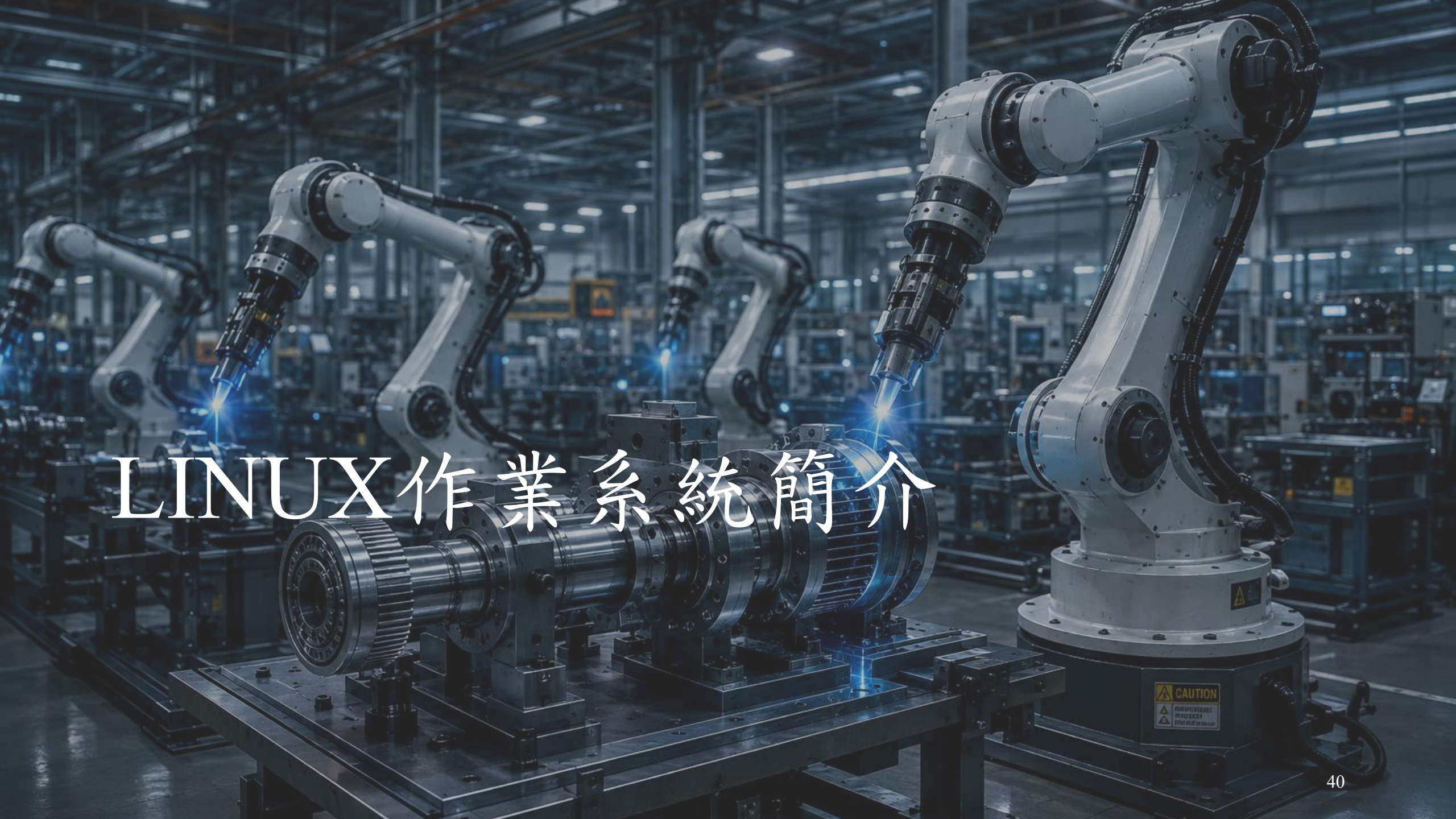




環境建立-安裝UBUNTU 18.04 LTS

- VMware設定
 - 直接啟動剛剛所建立好的虛擬機
 - 等待虛擬機的初始設定完成



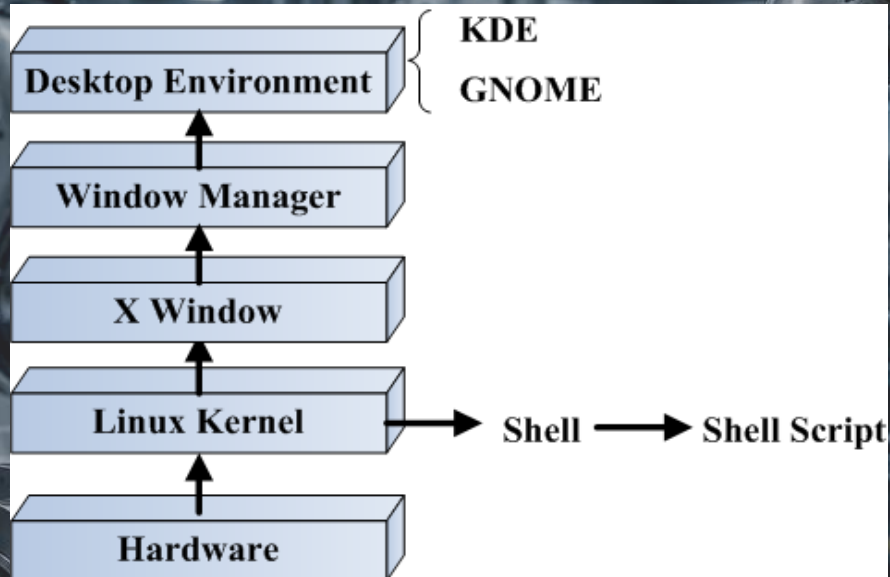
A photograph of a modern industrial factory floor. Several white robotic arms are visible, each equipped with a welding torch. They are positioned over a large, complex metal assembly that appears to be a part of a machine, possibly a motor or a pump. The background shows the vast, dimly lit interior of the factory with various structural elements and other equipment. The overall color palette is cool, dominated by blues and greys, with the bright blue sparks from the welding torches providing a focal point of light.

LINUX作業系統簡介



LINUX架構 (1)

- Linux架構：

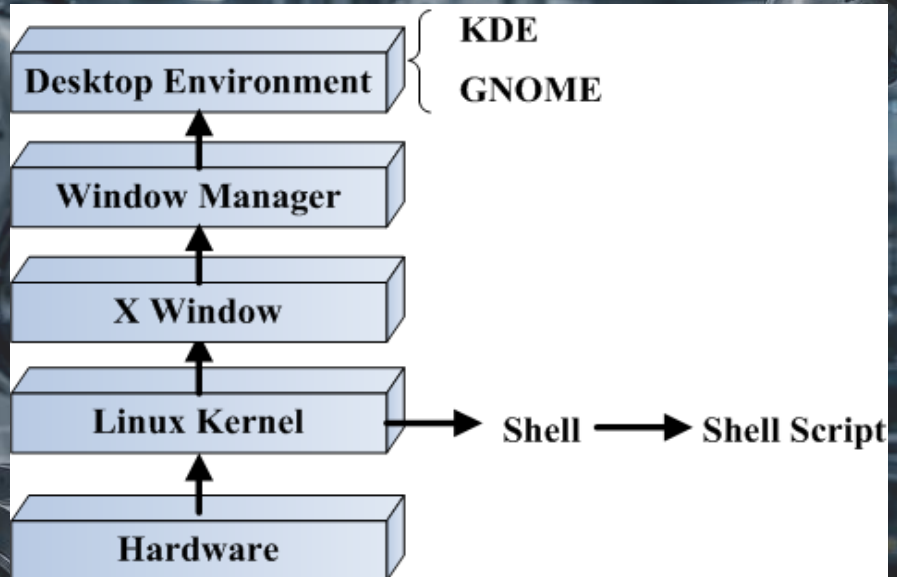


- Hardware：硬體設備
- Linux Kernel：用於控制硬體、管理程序、排程、檔案系統、I/O等等。
版本編號：偶數為穩定版(正式釋出)，奇數為開發版(測試中)



LINUX架構 (2)

- Linux架構：

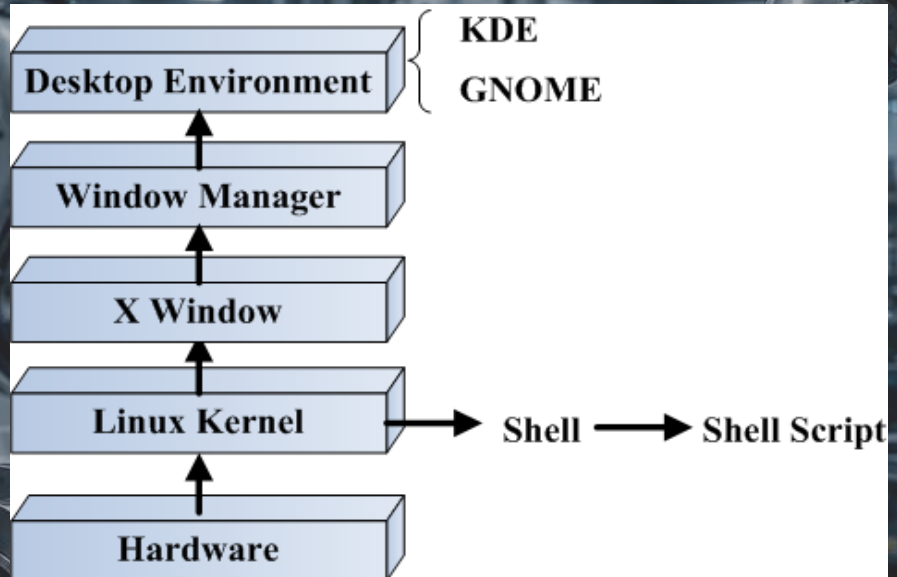


- Shell：讓使用者透過命令操作系統應用程式
- Shell Script：將各種命令操作寫成一個腳本，以利使用



LINUX架構 (3)

- Linux架構：



- X Window：OS上的應用程式，提供實現GUI基本功能
- Window Manager：管理應用X Window的GUI程式
- Desktop Environment：分為：GNOME以及KDE



檔案系統 (1)

- 檔案系統：

```
li@li: /  
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
(base) li@li:~$ cd /  
(base) li@li:/$ ls  
bin      dev      initrd.img  lib64      mnt      root     snap      sys      var  
boot     etc      initrd.img.old  lost+found  opt      run      srv        tmp      vmlinuz  
cdrom    home     lib         media      proc     sbin     swapfile  usr      vmlinuz.old  
(base) li@li:/$
```

- cd /: 代表移動到根目錄底下，ls: 列出現在位址有哪些資料，在此將列出較重要之目錄進行說明。
 - boot: 存放開機引導(boot loader)的檔案與開機選單(通常為grub)。
 - bin: 重要系統程式執行檔存放地點。
 - sbin: 存放系統管理員，會用到的指令。
 - dev: 存放裝置與週邊設備。
 - proc: 虛擬檔案系統，存放於記憶體中，不佔空間



檔案系統 (2)

• 檔案系統：

```
li@li: /  
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
(base) li@li:~$ cd /  
(base) li@li:/$ ls  
bin      dev      initrd.img  lib64      mnt      root     snap     sys      var  
boot     etc      initrd.img.old  lost+found  opt      run      srv      tmp      vmlinuz  
cdrom    home    lib         media      proc     sbin     swapfile  usr      vmlinuz.old  
(base) li@li:/$
```

- tmp：放暫存資料用
- etc：系統主要設定檔存放位址。
- lib：系統使用到的函式庫存放位址
- root：系統管理員(root)的目錄
- mnt：掛載設備用，例如：CD/DVD或USB
- media：掛載設備用，例如：CD/DVD或USB



檔案系統 (3)

• 檔案系統：

```
li@li: /  
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
(base) li@li:~$ cd /  
(base) li@li:/$ ls  
bin      dev      initrd.img      lib64      mnt      root     snap      sys      var  
boot     etc      initrd.img.old  lost+found opt       run      srv       tmp       vmlinuz  
cdrom    home    lib             media      proc     sbin     swapfile  usr       vmlinuz.old  
(base) li@li:/$
```

- `srv`：服務啟動後，所需之資料存放位址。
- `usr`：包含系統主要程式、圖形介面所需之檔案、函式庫等等。
- `home`：存放使用者家目錄，此目錄下會有與使用者帳號同名的資料夾。
- `opt`：額外安裝軟體所存放之位址。
- `var`：這是系統在工作時，預設的工作目錄，例如這架主機使用者的登錄檔案資訊、尚未寄出的郵件存放地、接收的郵件放置處等等。



檔案系統 (4)

- 檔案系統：

```
li@li: /  
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
(base) li@li:~$ cd /  
(base) li@li:/$ ls  
bin    dev    initrd.img    lib64    mnt    root    snap    sys    var  
boot   etc    initrd.img.old  lost+found  opt    run    srv     tmp    vmlinuz  
cdrom  home  lib           media     proc   sbin   swapfile  usr    vmlinuz.old  
(base) li@li:/$
```

- sys：功能與proc類似。
- lost+found：使用ext2/ext3/ext4檔案格式所出現的目錄，用於檔案系統發生錯誤時，存放遺失的檔案。



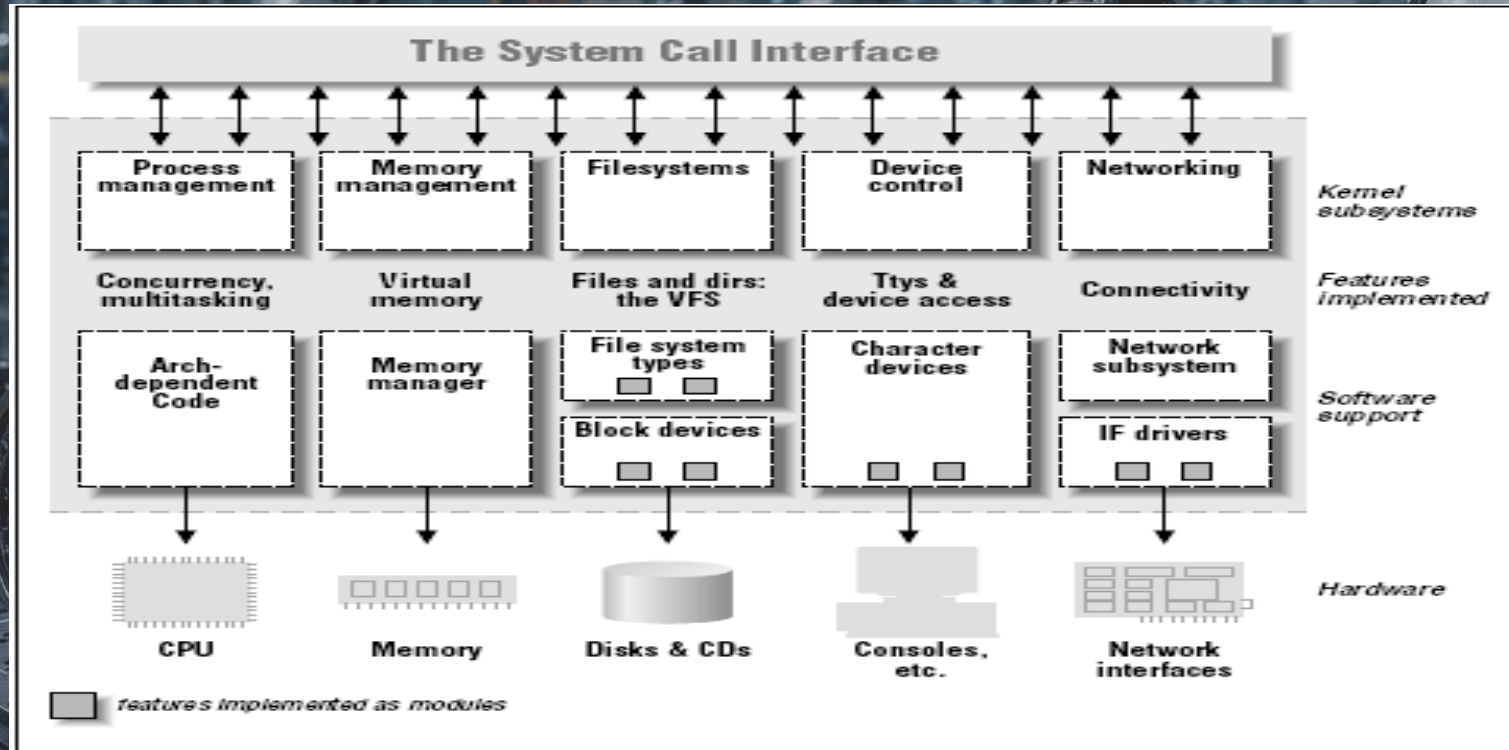
THE LINUX KERNEL

- Task mangement
- System calls
- Memory mangement
- Scheduler
- IPC
- File system
- Device driver
- Network stack
- SMP support



LINUX KERNEL ARCHITECTURE

- Linux Kernel architecture of a generic Linux system





EXECUTION MODE

- User mode
- Kernel mode
- System interface

User Applications

O/S Services

Linux Kernel

Hardware
Controllers



FROM USER TO KERNEL

- User level
 - User Program
 - Compiling
 - Linking
 - Execution
- Kernel
 - Loading program
 - System call
 - Device driver



LINUX-基本指令介紹

- 在這節，將簡單介紹比較常用的Linux Command，在開啟終端機，輸入指令即可得到相對應的功能。
- 介紹Linux檔案權限管理，以了解各種檔案與資料夾對於每個使用者皆有各個不一樣的使用權限。



LINUX-基本指令介紹

- man：透過man可查詢指令用法與用途，按q離開
- ls：顯示檔案資訊以及資料夾
 - -l：此參數顯示詳細資訊
 - -a：此參數顯示資訊，包含隱藏檔案
 - -h：此參數將檔案容量以人類較易讀的方式(例如 GB,MB,KB 等等)列出來
- cd：移動到資料夾
 - cd ~：移動到家目錄
 - cd ..：移動到上一個目錄
 - cd <name>：移動到<name>資料夾
- cp：複製檔案(cp 欲複製之檔案名稱 輸出地點)
 - -r：此參數遞迴持續複製，用於目錄的複製行為
- rm：刪除檔案的指令 (rm <檔案名稱>)
 - -f：此參數是 force 的意思，忽略不存在的檔案，不會出現警告訊息；
 - -r：此參數為遞迴刪除啊，最常用在目錄的刪除，這是非常危險的選項！
- mv：移動檔案到指定地點、修改檔案名稱(mv <檔案名稱> <地點>)
- mkdir：建立新資料夾
- rmdir：刪除空的資料夾



LINUX-基本指令介紹

- df：回報各個掛載之檔案系統使用情況
- ln：建立連結給其他檔案(ln -s <原始檔案或位置> <捷徑名稱>)
- shutdown：關機
 - -h：此參數為關機
 - -r：此參數為重新開機
- reboot：重新開機
- pwd：回報目前所在的目錄位址
- who：查看有哪些使用者在此系統上
- mount：掛載裝置至檔案系統上
- umount：從檔案系統上卸載裝置
- uname：顯示系統資訊(uname -a)
- passwd：改變密碼
- login, logout：登入登出系統



LINUX-基本指令介紹

- chown：更換檔案或目錄的擁有使用者與擁有群組
 - chown <使用者>:<群組> <檔案或目錄>
- chmod：更換檔案或目錄的存取權限
 - chmod ug+rx <目錄或檔案> (增加目錄或檔案的使用者與群組的讀取與執行權限)
 - chmod 755 <目錄或檔案> (將目錄或檔案的權限改為755)
- *權限內容於後面 Linux-權限管理 章節會再進一步說明
- mount：掛載裝置到系統
 - Ex.：mkdir /c (first make directory)
 - mount /dev/hdc /c
- umount：解除掛載裝置到系統
 - Ex.：umount /dev/hdc /c
- uname：顯示系統資訊 <ex> uname -a
- cat：查看ASCII文件內容 <ex> cat a.txt



LINUX-NETWORK

- ping : requests packet echos from network hosts
 - <ex> ping 140.124.182.5
- telnet : to access remote computers
 - <ex> telnet 140.124.182.5
- ssh : to access remote computers
 - <ex> <username>@<addresss(ip/domain name)>
- ifconfig : display the state of network interfaces setup network
- netstat : display network state
- ftp : ftp tool <ex> ftp 140.124.182.5
 - bin : binary mode of ftp
 - get filename : download file
 - put filename : upload file
 - bye : finish ftp
 - ls : list file and directory



LINUX-SEARCH

- `find` : `find [pathnames] [conditions]`
 - An extremely useful command for finding particular groups of files.
- `grep` : `grep [options] pattern [files]`
 - Search one or more files for lines that match a regular expression pattern.



LINUX-RESOURCE MANAGE

- `ps` : `ps [options]`
 - `-a` : list all processes on a terminal
 - `-u` : display processes for the specified users
 - `-x` : display processes without an associated terminal

```
miraculous@miraculous-desktop:~$ ps -aux
Warning: bad ps syntax, perhaps a bogus '-'? See http://procps.sf.net/faq.html
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.2  2528  1488 ?        Ss   Mar21    0:01 /sbin/init
root         2  0.0  0.0      0     0 ?        S<   Mar21    0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S<   Mar21    0:00 [migration/0]
root         4  0.0  0.0      0     0 ?        S<   Mar21    0:00 [ksoftirqd/0]
root         5  0.0  0.0      0     0 ?        S<   Mar21    0:00 [watchdog/0]
root         6  0.0  0.0      0     0 ?        S<   Mar21    0:00 [events/0]
root         7  0.0  0.0      0     0 ?        S<   Mar21    0:00 [cpuset]
root         8  0.0  0.0      0     0 ?        S<   Mar21    0:00 [khelper]
root         9  0.0  0.0      0     0 ?        S<   Mar21    0:00 [netns]
root        10  0.0  0.0      0     0 ?        S<   Mar21    0:00 [async/mgr]
root        11  0.0  0.0      0     0 ?        S<   Mar21    0:00 [kintegrityd/0]
root        12  0.0  0.0      0     0 ?        S<   Mar21    0:00 [kblockd/0]
root        13  0.0  0.0      0     0 ?        S<   Mar21    0:00 [kacpid]
root        14  0.0  0.0      0     0 ?        S<   Mar21    0:00 [kacpi_notify]
root        15  0.0  0.0      0     0 ?        S<   Mar21    0:00 [kacpi_hotplug]
root        16  0.1  0.0      0     0 ?        S<   Mar21    1:57 [ata/0]
root        17  0.0  0.0      0     0 ?        S<   Mar21    0:00 [ata_aux]
root        18  0.0  0.0      0     0 ?        S<   Mar21    0:00 [ksuspend_usbd]
root        19  0.0  0.0      0     0 ?        S<   Mar21    0:00 [khubd]
root        20  0.0  0.0      0     0 ?        S<   Mar21    0:00 [kseriod]
```

- `Kill` : `kill [options] [pid] [command]`
 - Send a signal to terminate one or more process IDs.



LINUX-權限管理

- 檔案權限：
 - Read：讀取
 - Write：寫入
 - Execute：執行
- 使用者分類：
 - 系統管理員(root)：為最上層管理者，可取得一切資訊
 - 使用者(user)：單一使用者所建立之帳號
 - 群組(group)：若很多使用者，可歸類為一個群組，則可讓user同時加入一個group，並針對該group開啟權限。
 - 其他(other)：其他拜訪者。



檔案權限 (1)

- 在終端機下輸入 `ls -al` 列出該路徑下所有檔案之屬性內容，其格式輸出將會呈現：

目錄	r:代表讀取權限								
檔案	w:代表寫入權限								
連結	x:代表執行權限								
	- :無此權限								
[]	rwX	rwX	rwX	n	root	root	4096	yyyy-mm-dd hh:mm	檔名
	擁有人的權限	同群組的權限	其他人的權限	連結數	檔案擁有者	檔案所屬群組	檔案容量	檔案最後修改時間	

- [d] : 目錄
- [-] : 檔案
- [l] : 連結檔
- [b] : 可供儲存的週邊設備
- [c] : 序列埠設備，如鍵盤、滑鼠等



檔案權限 (2)

- 在執行上述指令後，以下是在根目錄下所得到的結果，在bin中，為drwxr-xr-x，代表他為一個資料夾，而擁有人的權限為可讀、可寫、可執行，同群組的權限可讀、可執行，其他人的權限為可讀、可執行。

```
(base) li@li:/$ ls -al
總計 2097328
drwxr-xr-x  24 root root    4096 二  2 06:32 .
drwxr-xr-x  24 root root    4096 二  2 06:32 ..
drwxr-xr-x   2 root root    4096 十一 30 06:08 bin
drwxr-xr-x   4 root root    4096 十二  3 06:10 boot
drwxrwxr-x   2 root root    4096 十四 14 2022 cdrom
drwxr-xr-x  19 root root   4900 十四 17 15:09 dev
drwxr-xr-x 141 root root  12288 十四  2 06:34 etc
drwxr-xr-x   3 root root    4096 十四 14 2022 home
lrwxrwxrwx   1 root root     33  二  2 06:32 initrd.img -> boot/initrd.img-5.4.0-137-generic
lrwxrwxrwx   1 root root     33  二  2 06:32 initrd.img.old -> boot/initrd.img-5.4.0-135-generic
drwxr-xr-x  22 root root    4096 十一  9 20:08 lib
drwxr-xr-x   2 root root    4096 十一  9 20:08 lib64
drwx-----  2 root root  16384 十四 14 2022 lost+found
drwxr-xr-x   3 root root    4096 十四 14 2022 media
drwxr-xr-x   2 root root    4096 十四  4 2020 mnt
drwxr-xr-x   5 root root    4096 十四 18 2022 opt
```



檔案權限 (3)

- 執行 `cd ~`，進入自己帳號路徑，輸入 `ls -al`。
- 可以發現大部份檔案的擁有人以及群組，為自己帳號。

```
(base) li@li:~$ cd ~
(base) li@li:~$ ls -al
總計 2949940
drwxr-xr-x 38 li  li      4096 二  22 19:15 .
drwxr-xr-x  3 root root    4096 二  14 2022 ..
drwxrwxr-x 11 li  li      4096 九   1 20:38 old_versions
-rw-rw-r--  1 li  li      222 十一  8 18:32 aaa
-rw-----  1 li  li    69188 十二 22 19:15 .bash_history
-rw-r--r--  1 li  li     220 二  14 2022 .bash_logout
-rw-rw-r--  1 li  li    4483 十一 15 16:12 .bashrc
drwx----- 31 li  li     4096 十一  9 17:16 .cache
drwx-----  2 li  li     4096 十一 23 14:35 .chewing
drwxrwxr-x  2 li  li     4096 十一 15 16:01 .conda
drwx----- 30 li  li     4096 十二 20 12:23 .config
drwx-----  3 root root    4096 十二 14 2022 .dbus
drwxr-xr-x  5 li  li     4096 十二 23 12:17 Desktop
drwxr-xr-x  2 li  li     4096 十一 21 17:20 Documents
drwxrwxr-x  3 li  li     4096 十二  6 14:20 .dotnet
drwxr-xr-x  4 li  li     4096 十二 22 15:30 Downloads
-rw-r--r--  1 li  li    8980 二  14 2022 examples.desktop
-rw-rw-r--  1 li  li     225 二  23 2022 .gitconfig
drwx-----  3 li  li     4096 二  9 2022 .gnome
drwx-----  3 li  li     4096 二  8 2022 .gnupg
drwxrwxr-x  2 li  li     4096 一 10 00:14 .gphoto
```



檔案權限 (4)

- 當在屬於自己權限的路徑底下時，進行讀檔、寫檔、執行等動作，皆不會進行使用者確認，因為該資料夾的擁有人屬於執行這些權限的人。

```
(base) li@li:~$ cd ~  
(base) li@li:~$ mkdir mDoc  
(base) li@li:~$ ls  
old_versions  Documents      mDoc  
aaa           Downloads      miniconda3  
Desktop       examples.desktop  Music
```

- 進入不屬於自己權限的路徑底下時，進行進行讀檔、寫檔、執行等動作，皆因為權限不足而遭受系統拒絕，此時可利用root的權限進行這些動作。
- sudo 指令可讓使用者暫時取得root的權限進行操作。

```
(base) li@li:~$ cd /home  
(base) li@li:/home$ mkdir mAccount  
mkdir: 無法建立目錄'mAccount': 拒絕不符權限的操作  
(base) li@li:/home$ sudo mkdir mAccount  
[sudo] password for li:  
(base) li@li:/home$ ls  
li mAccount  
(base) li@li:/home$
```



檔案權限 (5)

- 什麼是UID與GID
 - UID與GID為系統主要運作基礎，幫助識別用戶權限。
 - UID：userid
 - GID：groupid
- 使用者密碼資料，存放於/etc/passwd
- 映射資料，存放於/etc/shadow
 - 由於每個程序都需要透過UID，GID判斷權限，故在passwd的權限則為-rw-r--r--，如此將會造成任何人都可看到每個人的密碼，故將密碼移入到shadow並加密，透過映射方式進行存取，以增加帳戶安全性。
- 使用者群組資料，存放於/etc/group



檔案權限 (6)

- 如何增加與移除使用者以及群組：

- adduser：增加使用者
- deluser：刪除使用者
- groupadd：增加群組
- groupdel：刪除群組
- gpasswd：設定群組與密碼

- 如何改變檔案權限：

- chgrp：改變檔案所屬群組
- chown：改變檔案所屬人
- chmod：改變檔案的屬性



檔案權限 (7) - /ETC/PASSWD

- 1. UID
- 2. GID
- 3. 一般資料
- 4. 家目錄
- 5. 和系統溝通所使用的Shell



檔案權限 (8) - /ETC/SHADOW

- / etc/shadow

```
li@li: ~  
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
sync:*:18295:0:99999:7:::  
games:*:18295:0:99999:7:::  
man:*:18295:0:99999:7:::  
lp:*:18295:0:99999:7:::  
mail:*:18295:0:99999:7:::  
news:*:18295:0:99999:7:::  
uucp:*:18295:0:99999:7:::  
proxy:*:18295:0:99999:7:::  
www-data:*:18295:0:99999:7:::  
backup:*:18295:0:99999:7:::  
l1st:*:18295:0:99999:7:::  
irc:*:18295:0:99999:7:::  
gnats:*:18295:0:99999:7:::  
nobody:*:18295:0:99999:7:::  
systemd-network:*:18295:0:99999:7:::  
systemd-resolve:*:18295:0:99999:7:::  
syslog:*:18295:0:99999:7:::  
messagebus:*:18295:0:99999:7:::  
_apt:*:18295:0:99999:7:::  
u1dd:*:18295:0:99999:7:::  
avahi-autoipd:*:18295:0:99999:7:::  
usbmux:*:18295:0:99999:7:::  
dnsmasq:*:18295:0:99999:7:::  
rtkit:*:18295:0:99999:7:::  
cups-pk-helper:*:18295:0:99999:7:::  
speech-dispatcher:!:18295:0:99999:7:::  
whoopsie:*:18295:0:99999:7:::  
kernoops:*:18295:0:99999:7:::  
saned:*:18295:0:99999:7:::  
pulse:*:18295:0:99999:7:::  
avahi:*:18295:0:99999:7:::  
colorc:*:18295:0:99999:7:::  
hplip:*:18295:0:99999:7:::  
geoclue:*:18295:0:99999:7:::  
gnome-initial-setup:*:18295:0:99999:7:::  
gdm:*:18295:0:99999:7:::  
li:$6$03uIpS3P$LRXRYUoDTxNi3AmaIzCIYxDVQujp6DD8tpEq64PFDE6hbGonyfU0jE72n0XCzqmLyTkZJ5jz0x1L/AKfpnaBe.:19096:0  
:99999:7:::  
sshd:*:19157:0:99999:7:::  
nvidia-persistenced:*:19311:0:99999:7:::
```



檔案權限 (9) - /ETC/SHADOW

- 資料格式，皆用：當區隔，共九個欄位
- 1. 使用者名稱
- 2. 加密後的密碼
- 3. 上次改密碼的時間(Linux日期以1970/01/01為起始)
- 4. 密碼經過幾天可以變更
- 5. 密碼經過幾天必須變更
- 6. 密碼過期之前幾天需警告擁有人
- 7. 密碼過期幾天後帳號會被取消
- 8. 帳號何時失效
- 9. 保留，目前沒用



檔案權限 (10) - /ETC/GROUP

- /etc/group

- 資料格式：

- 1. 群組名稱
- 2. 群組密碼

- 3. GID

- 4. 群組成員，使用“,”分隔

```
li@li: ~  
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
root:x:0:  
daemon:x:1:  
bin:x:2:  
sys:x:3:  
adm:x:4:syslog,li  
tty:x:5:  
disk:x:6:  
lp:x:7:  
mail:x:8:  
news:x:9:  
uucp:x:10:  
man:x:12:  
proxy:x:13:  
kmem:x:15:  
dialout:x:20:  
fax:x:21:  
voice:x:22:  
cdrom:x:24:li  
floppy:x:25:  
tape:x:26:  
sudo:x:27:li  
audio:x:29:pulse  
dip:x:30:li  
www-data:x:33:  
backup:x:34:  
operator:x:37:  
list:x:38:  
irc:x:39:  
src:x:40:  
gnats:x:41:  
shadow:x:42:  
utmp:x:43:  
video:x:44:  
sasl:x:45:  
plugdev:x:46:li  
staff:x:50:  
games:x:60:  
users:x:100:  
nogroup:x:65534:  
systemd-journal:x:101:  
"/etc/group" 68 lines, 939 characters
```



檔案權限 (11) - ADDUSER

- 當要增加使用者時，輸入
 - `sudo adduser` “帳戶名稱”之後進入設定帳號詳細資料
 - 帳戶僅接受小寫

```
(base) li@li:~$ sudo adduser mlinux
正在新增使用者 'mlinux' ...
增加新群組 'mlinux' (1001) ...
正在新增新使用者 'mlinux' (UID 1001) 到群組 'mlinux' ...
正在新增家目錄 '/home/mlinux' ...
正在從 '/etc/skel' 複製檔案 ...
輸入新的 UNIX 密碼：
再次輸入新的 UNIX 密碼：
passwd: 密碼已成功地變更
正在改變 mlinux 的使用者訊息
請輸入新值，或直接按 ENTER 鍵以使用預設值
全名 []: mName
房間號碼 []: mFName
工作電話 []: mPhone
住家電話 []: mPhone
其它 []: mOther
以上輸入的資料正確嗎？[Y/n] y
(base) li@li:~$
```



檔案權限 (12) - ADDUSER

- 在新增完使用者帳戶後，可經由/etc/passwd進行確認是否有新增成功。



```
pulse:x:115:120:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
avahi:x:116:122:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
colord:x:117:123:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
hplip:x:118:7:HPLIP system user,,,:/var/run/hplip:/bin/false
geoclue:x:119:124:/:/var/lib/geoclue:/usr/sbin/nologin
gnome-initial-setup:x:120:65534:/:/run/gnome-initial-setup:/bin/false
gdm:x:121:125:Gnome Display Manager:/var/lib/gdm3:/bin/false
li:x:1000:1000:li,,,:/home/li:/bin/bash
sshd:x:123:65534:/:/run/sshd:/usr/sbin/nologin
nvidia-persistenced:x:122:127:NVIDIA Persistence Daemon,,,:/nonexistent:/usr/sbin/nologin
mlinux:x:1001:1001:mName,mFName,mPhone,mPhone,mOther:/home/mlinux:/bin/bash
```

- 其資料對應前章所提格式。



檔案權限 (13) - DELUSER

- 當要移除使用者時，則輸入
- `sudo deluser “帳戶名稱”`

```
(base) li@li:~$ sudo deluser mlinux
正在移除使用者 'mlinux' ...
Warning: group 'mlinux' has no more members.
完成。
```



```
cups-pk-helper:x:110:110:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin
speech-dispatcher:x:111:29:Speech Dispatcher,,,:/var/run/speech-dispatcher:/bin/false
whoopsie:x:112:117::/nonexistent:/bin/false
kernoops:x:113:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
saned:x:114:119::/var/lib/saned:/usr/sbin/nologin
pulse:x:115:120:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
avahi:x:116:122:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
colord:x:117:123:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
hplip:x:118:7:HPLIP system user,,,:/var/run/hplip:/bin/false
geoclue:x:119:124::/var/lib/geoclue:/usr/sbin/nologin
gnome-initial-setup:x:120:65534::/run/gnome-initial-setup:/bin/false
gdm:x:121:125:Gnome Display Manager:/var/lib/gdm3:/bin/false
E486: Pattern not found: mlinux
```



檔案權限 (14) - GROUPADD

- 新增一個群組：
- `sudo groupadd` “群組名稱”

```
(base) li@li:~$ sudo addgroup mgroup
[sudo] password for li:
新增 'mgroup' 群組 (GID 1001) ...
完成。
(base) li@li:~$
```



```
li@li: ~
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)
gnats:x:41:
shadow:x:42:
utmp:x:43:
video:x:44:
sasl:x:45:
plugdev:x:46:li
staff:x:50:
games:x:60:
users:x:100:
nogroup:x:65534:
systemd-journal:x:101:
systemd-network:x:102:
systemd-resolve:x:103:
input:x:104:
crontab:x:105:
syslog:x:106:
messagebus:x:107:
netdev:x:108:
mlocate:x:109:
ssl-cert:x:110:
uucp:x:111:
avahi-autoipd:x:112:
bluetooth:x:113:
rtkit:x:114:
ssh:x:115:
lpadmin:x:116:li
whoopsie:x:117:
scanner:x:118:saned
saned:x:119:
pulse:x:120:
pulse-access:x:121:
avahi:x:122:
colord:x:123:
geoclue:x:124:
gdm:x:125:
li:x:1000:
smbshare:x:126:li
nvidia-persistenced:x:127:
rdma:x:128:
mgroup:x:1001:
```



檔案權限 (15) - GROUPDEL

- 刪除一個群組：
- `sudo groupdel “群組名稱”`

```
(base) li@li:~$ sudo groupdel mgroup
```



```
shadow:x:42:  
utmp:x:43:  
video:x:44:  
sasl:x:45:  
plugdev:x:46:li  
staff:x:50:  
games:x:60:  
users:x:100:  
nogroup:x:65534:  
systemd-journal:x:101:  
E486: Pattern not found: mgroup
```



檔案權限 (16) - GPASSWD

- 當我們想要把使用者設定為某一群組成員時，可以透過gpasswd的指令進行設定，除了設定群組成員外，也可以進行群組密碼的設定。
 - gpasswd “群組名稱”：指定該群組密碼
 - gpasswd [-A user, ...] “群組名稱”：指定user為群組管理員
 - gpasswd [-M user, ...] “群組名稱”：將user加入該群組
 - gpasswd [-r] “群組名稱”：將群組密碼移除
- 現在將之前建立的帳戶，加入到新增群組之中
 - sudo gpasswd -M mlinux mgroup

```
(base) li@li:~$ sudo gpasswd -M li mgroup  
(base) li@li:~$
```



```
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)  
mgroup:x:1001:li
```



檔案權限 (17) - CHGRP

- chgrp : 改變所屬群組，而所改變的群組，必須要在/etc/group裡所存在的名稱。
 - sudo chgrp [-R] “群組名稱” “改變的檔案或資料夾”
 - [-R] 參數：代表是否持續改變該資料夾底下所有檔案

```
(base) li@li:~$ cd /home
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:31 .
drwxr-xr-x 24 root root   4096  2  06:32 ..
drwxr-xr-x 39 li  li     4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgrou 4096  23 16:31 mlinux
(base) li@li:/home$ sudo chgrp mgrou li
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:31 .
drwxr-xr-x 24 root root   4096  2  06:32 ..
drwxr-xr-x 39 li  mgrou 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgrou 4096  23 16:31 mlinux
(base) li@li:/home$
```



檔案權限 (18) - CHOWN

- **chown**：改變檔案所有人，而所改變的擁有者，必須是在 `/etc/passwd` 中所紀錄的使用者名稱。
 - `sudo chown [-R]` “帳號名稱” “檔案或資料夾”
 - `[-R]` 參數：代表是否持續改變該資料夾底下所有檔案

```
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  2  06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgroup 4096  23 16:31 mlinux
-rw-r--r--  1 root root     0  23 16:47 mtest
(base) li@li:/home$ sudo chown li mtest
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  2  06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgroup 4096  23 16:31 mlinux
-rw-r--r--  1 li  root     0  23 16:47 mtest
(base) li@li:/home$
```



檔案權限 (19) - CHMOD

- `chmod`：改變檔案屬性，修改擁有者、群組、其他人使用檔案或資料夾的權限。其屬性對照如下：
 - `r`：4
 - `w`：2
 - `x`：1
- 檔案的權限，如果是777，則代表擁有者、群組、其他人，皆可對該檔案進行讀、且、執行的權限。
- 而修改方式為：
 - `chmod [-R] {i} {j} {k}` “檔案或目錄”
 - `[-R]` 參數：代表是否持續改變該資料夾底下所有檔案
 - `{i} {j} {k}`：分別代表各個使用者權限計算後之結果。



檔案權限 (20) - CHMOD

- 另外一個修改方式，則是將該群組分類，如下表：

chmod	u g o a	+ (加入) - (除去) = (設定)	r w x	檔案或目錄
-------	------------------	----------------------------	-------------	-------

- u : user
- g : group
- o : other
- a : all



檔案權限 (21) - CHMOD

- `sudo chmod 761 mtest`，擁有者擁有全部權限，群組擁有讀取與寫入權限，其他擁有執行權限。

```
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  23 06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgroup 4096  23 16:31 mlinux
-rw-r--r--  1 li  root      0  23 16:47 mtest
(base) li@li:/home$ sudo chmod 761 mtest
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  23 06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgroup 4096  23 16:31 mlinux
-rwxrwx---  1 li  root      0  23 16:47 mtest
(base) li@li:/home$
```



檔案權限 (22) - CHMOD

- `sudo chmod u=rwx,g=r,o=x mtest`，「`u=rwx,g=r,o=x`」中間並無任何空白，使mtest檔案，擁有者有讀取、寫入及執行權限，群組擁有讀取權限，其他擁有執行權限。

```
(base) li@li:~$ cd /home
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  23 06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgroup 4096  23 16:31 mlinux
-rwxrw---x  1 li  root      0  23 16:47 mtest
(base) li@li:/home$ sudo chmod u=rwx,g=r,o=x mtest
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  23 06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgroup 4096  23 16:31 mlinux
-rwxr---x  1 li  root      0  23 16:47 mtest
(base) li@li:/home$
```



檔案權限 (23) - CHMOD

- `sudo chmod 777 mlinux`，使mlinux所有使用者擁有全部權限。

```
(base) li@li:~$ cd /home
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  2  06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxr-xr-x  3 1001 mgroup 4096  23 16:31 mlinux
-rwxr---x  1 li  root      0  23 16:47 mtest
(base) li@li:/home$ sudo chmod 777 mlinux
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root   4096  23 16:47 .
drwxr-xr-x 24 root root   4096  2  06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root   4096  23 16:26 mAccount
drwxrwxrwx  3 1001 mgroup 4096  23 16:31 mlinux
-rwxr---x  1 li  root      0  23 16:47 mtest
(base) li@li:/home$
```



檔案權限 (24) - CHMOD

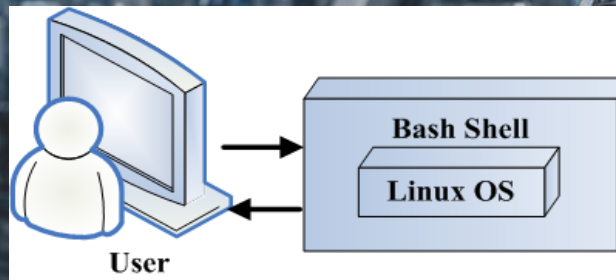
- `sudo chmod a=rwx mtest`，使mtest所有使用者擁有全部權限。

```
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root  4096  23 16:47 .
drwxr-xr-x 24 root root  4096  2  06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root  4096  23 16:26 mAccount
drwxrwxrwx  3 1001 mgroup 4096  23 16:31 mlinux
-rwxr----x  1 li  root    0  23 16:47 mtest
(base) li@li:/home$ sudo chmod a=rwx mtest
(base) li@li:/home$ ls -al
總計 20
drwxr-xr-x  5 root root  4096  23 16:47 .
drwxr-xr-x 24 root root  4096  2  06:32 ..
drwxr-xr-x 39 li  mgroup 4096  23 16:24 li
drwxr-xr-x  2 root root  4096  23 16:26 mAccount
drwxrwxrwx  3 1001 mgroup 4096  23 16:31 mlinux
-rwxrwxrwx  1 li  root    0  23 16:47 mtest
(base) li@li:/home$
```



SHELL

- 使用者與作業系統之間溝通的橋樑，當使用者輸入指令時，將轉換成為作業系統所知道的指令進行溝通。



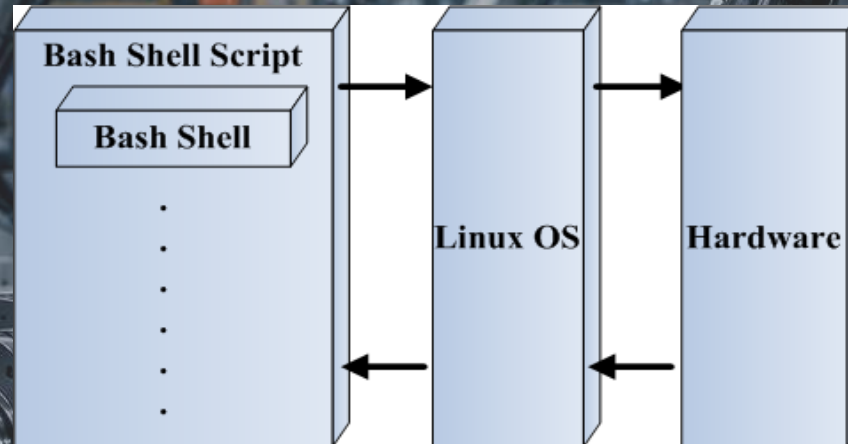
- 例如：cd，代表切換路徑，ls，代表列出該路徑下所有的檔案。

```
li@li: /
檔案(F) 編輯(E) 檢視(V) 搜尋(S) 終端機(T) 求助(H)
(base) li@li:~$ cd /
(base) li@li:/$ ls
bin    dev    initrd.img    lib64    mnt    root    snap    sys    var
boot   etc    initrd.img.old  lost+found  opt    run    srv     tmp    vmlinuz
cdrom  home  lib           media     proc   sbin   swapfile  usr    vmlinuz.old
(base) li@li:/$
```



SHELL SCRIPT (1)

- 可透過Shell Script建立指令腳本，方便使用者進行大量Shell指令與作業系統進行溝通



- 在每個Script中皆有他特別的意義存在，例如：作業系統啟動流程，也靠著Script進行一連串的系统啟動服務



SHELL SCRIPT (2)

- 撰寫一個簡單的Shell Script：

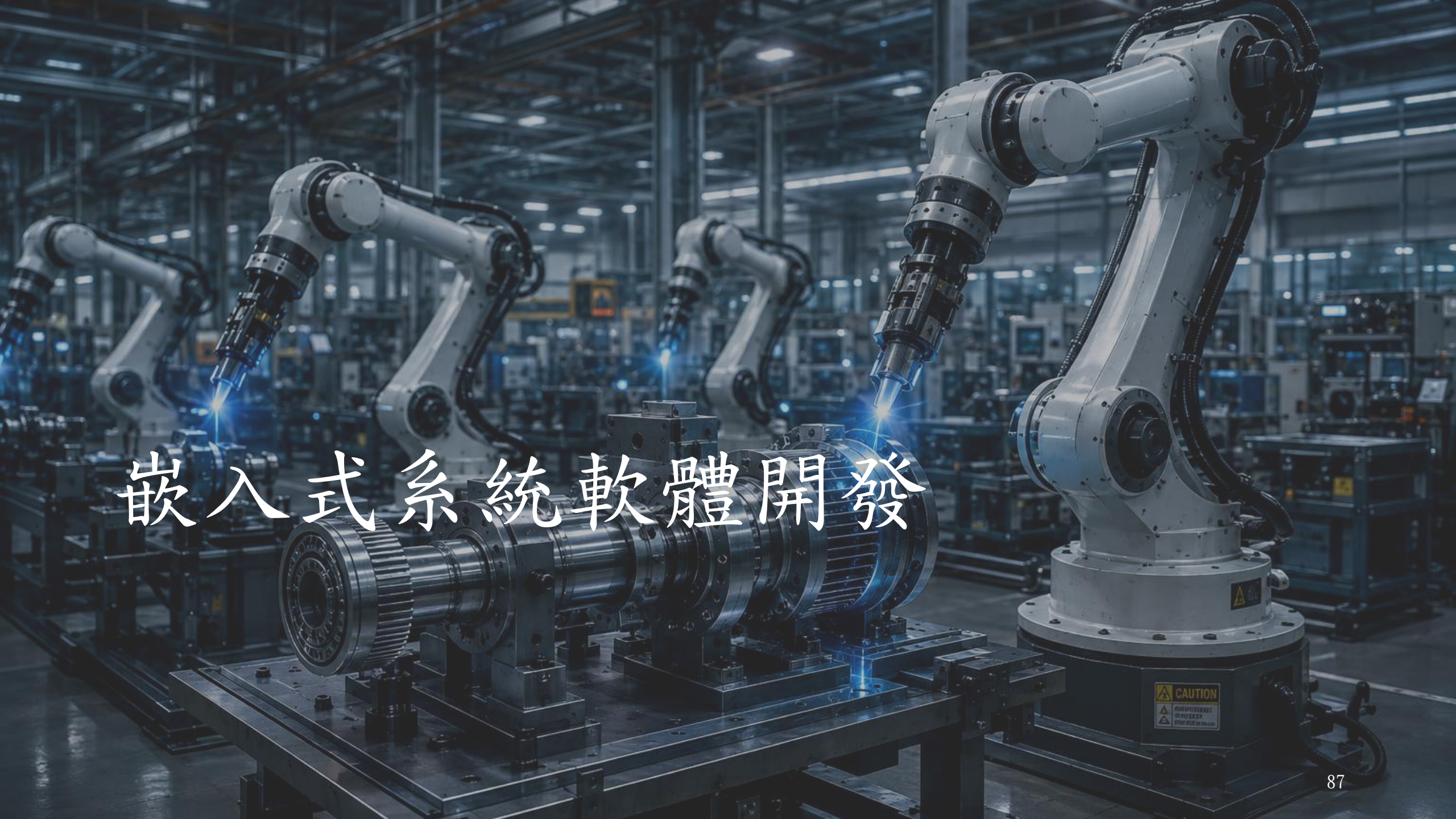
```
#!/bin/bash
```

```
cd /  
ls
```

切換到根目錄
顯示該目錄所有檔案

- 將該檔案加入執行權限：
 - `sudo chmod +x mShellScript`
- 執行輸出

```
(base) li@li:~$ cd ~  
(base) li@li:~$ sudo ./mShellScript  
bin    dev    initrd.img    lib64    mnt    root    snap    sys    var  
boot   etc    initrd.img.old  lost+found  opt    run    srv    tmp    vmlinuz  
cdrom  home  lib           media    proc   sbin   swapfile  usr    vmlinuz.old  
(base) li@li:~$
```

The background image shows a modern industrial manufacturing environment. Several white robotic arms are positioned around a large, complex metal assembly that is being welded. Bright blue sparks are visible at the points of contact between the welding torches and the metal. The scene is dimly lit, with the primary light source being the intense blue of the welding process. The overall atmosphere is one of precision and automation.

嵌入式系統軟體開發



嵌入式系統軟體開發

EMBEDDED SOFTWARE VS. NONEMBEDDED SOFTWARE

- Development technologies
 - Hard timing constraints
 - Limited memory resources and power usage
 - Predefined hardware platform technology
 - Related cost drivers of the hardware
 - Reliability, cost and time-to-market demands



EMBEDDED SOFTWARE VS. NON-EMBEDDED SOFTWARE

- Embedded software usually packaged with hardware product for selling
- Embedded software engineering and other processes such as mechanical engineering and electrical engineering are in fact sub-processes of system engineering.
- Coordinating these sub-processes to develop quality products is one of the embedded software development's most challenging aspects.

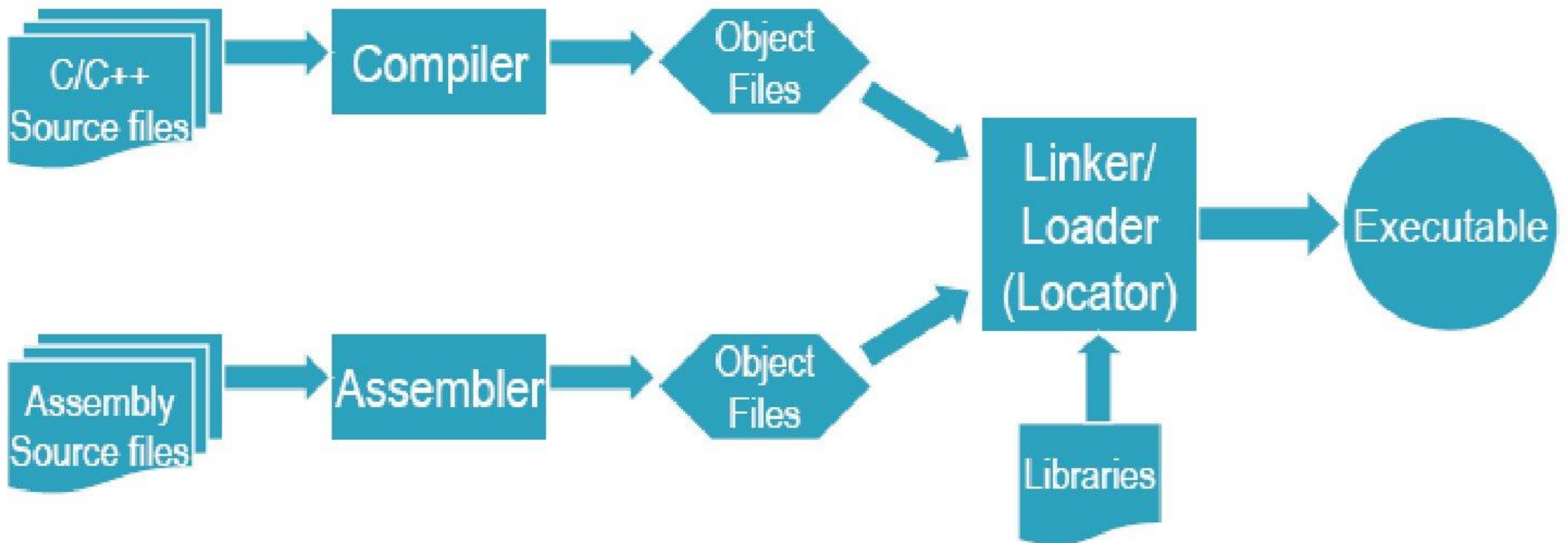


SOFTWARE DEVELOPMENT FOR DESKTOP COMPUTER SYSTEMS

- Typically software development for desktop computer systems are usually done on the same machine (Native development):
 - Create/edit the source code
 - Compile the source code to create executable program
 - Execute the code
 - OS creates the process and loads into memory
 - OS transfers control to the process and the process begins execution



CREATING EXECUTABLE CODE





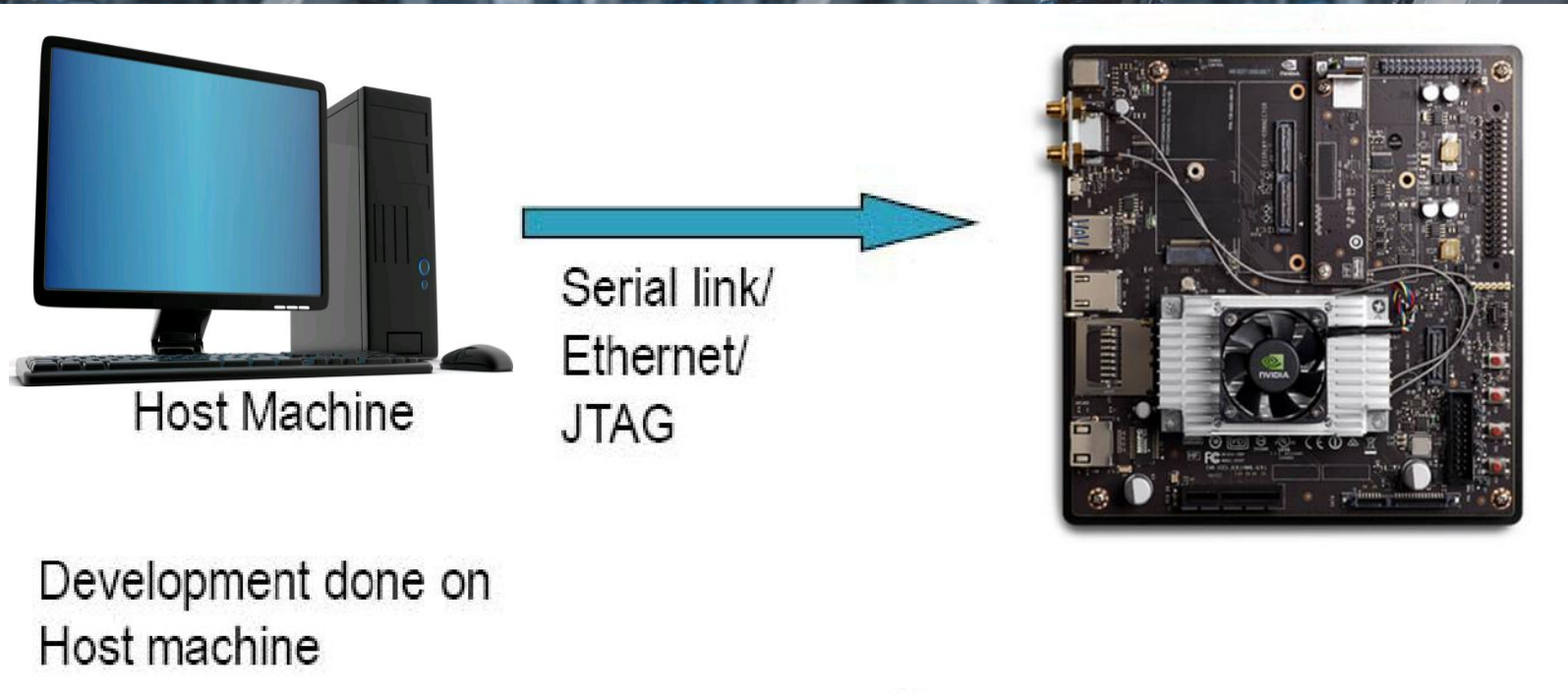
EMBEDDED SOFTWARE DEVELOPMENT

- The above process will not work exactly the same way for embedded systems:
 - Application programs are typically developed, compiled, and run on the host system
 - Embedded software is targeted for execution on a target processor(different from the development/host processor and operating environment)
 - We cannot directly develop the applications on the target hardware
 - In a general purpose computer environment, the OS and application software are distinct, but in embedded systems everything is often a single piece of code
 - Memory has to be explicitly managed (often no memory management/virtual memory)



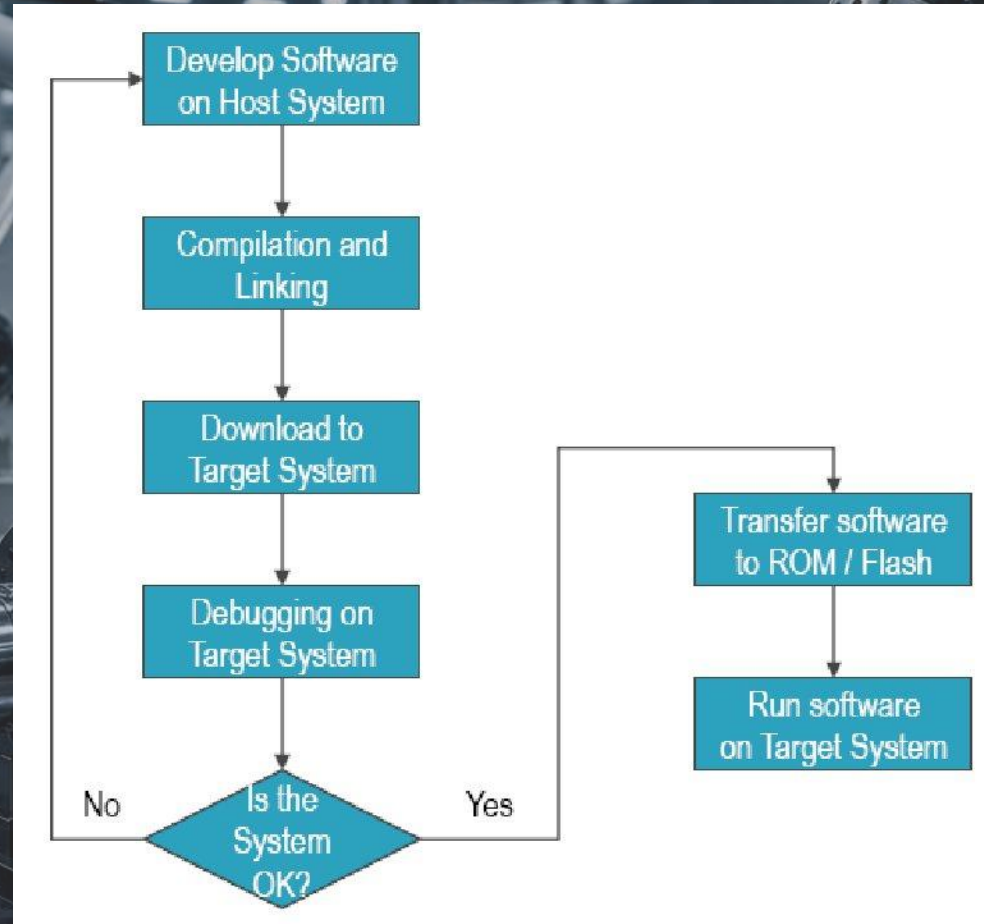
EMBEDDED SOFTWARE DEVELOPMENT

- Cross-platform development





EMBEDDED SOFTWARE DEVELOPMENT



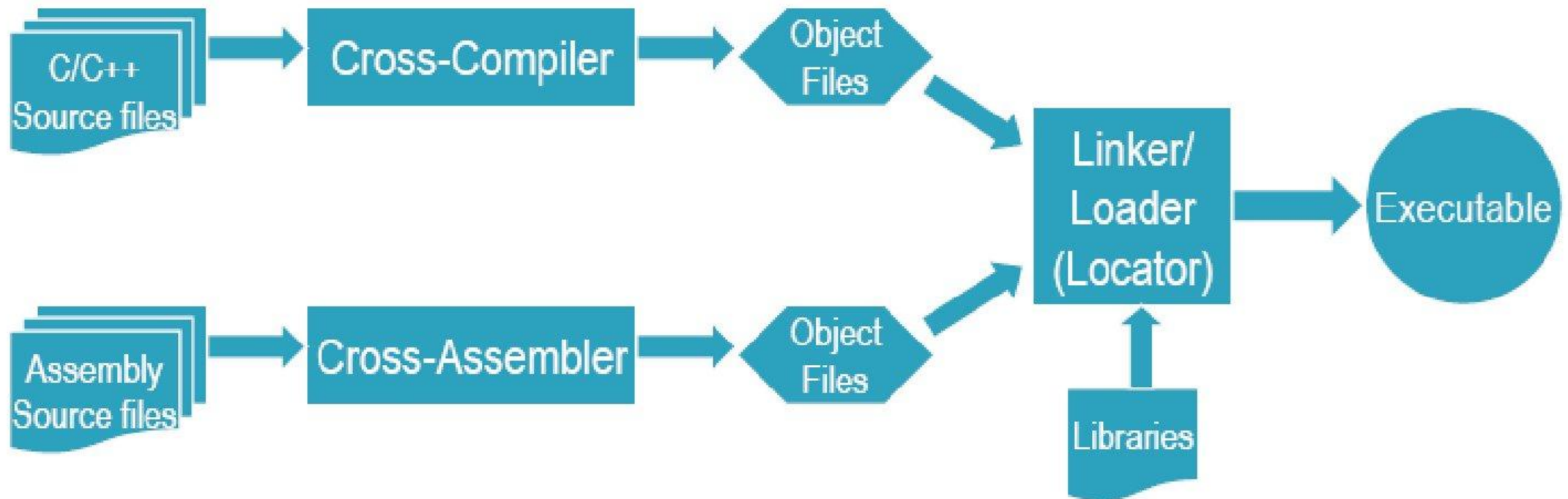


EMBEDDED SOFTWARE DEVELOPMENT TOOLS

- What tools are needed to develop, test, and locate embedded software into the target processor and its operating environment?
- Host: Where the embedded software is developed, compiled, tested, debugged, and optimized prior to its translation into target device.
 - Because the host has keyboards, editors, monitors, printers, more memory, etc. for development, while the target may not have these capabilities for developing the software.
- Target: After development, the code is cross-compiled, translated – cross-assembled, linked (into target processor instruction set) and located into the target



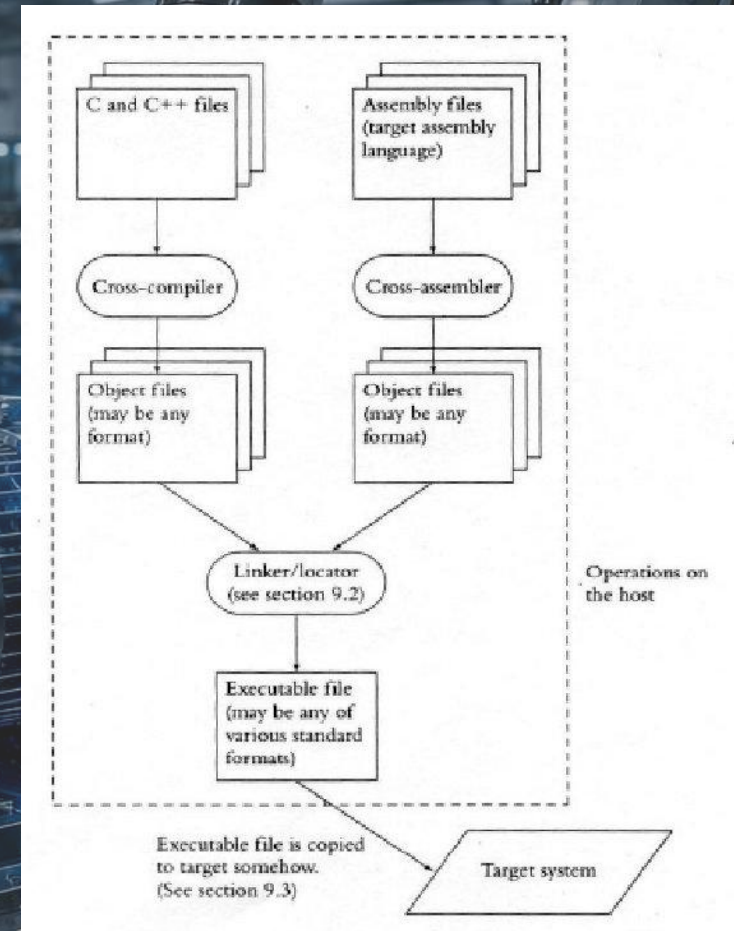
CREATING EXECUTABLE CODE FOR EMBEDDED SYSTEM





TOOLS CHAIN FOR BUILDING EMBEDDED SOFTWARE

- Cross-compiler
- Cross-assembler
- Cross-linker
- Cross-debugger
- Cross-compiled libraries
- OS dependent libraries
- and headers for target processor





GCC與MAKEFILE簡介

- 介紹GNU Compiler Collection，及其相關參數及使用方式，以了解其使用情況。
- 接著將介紹Makefile，可將一系列繁雜的編譯過程寫成一個Makefile檔，並透過指令自動執行編譯。



GCC 簡介

- gcc : GNU project C and C++ compiler
- Features of GNU cc :
 - The compilation process includes up to four stages :
 - - Preprocessing
 - - Compilation Proper
 - - Assembly
 - - Linking



INTERPRETS FILENAME EXTENSIONS

Extension	Type
.c	C language source code
.C, .cc	C++ language source code
.i	Preprocessed C source code
.ii	Preprocessed C++ source code
.S, .s	Assembly language source code
.o	Compiled object code
.a, .so	Compiled library code



範例

- Hello world:

```
#include <stdio.h>

int main(void)
{
    print("Hello World!\n");
    return 0;
}
```

```
root@miraculous-desktop:~# vim hello.c
root@miraculous-desktop:~# gcc hello.c -o hello
root@miraculous-desktop:~# ./hello
Hello World!
root@miraculous-desktop:~#
```



GCC COMMAND-LINE OPTIONS

- `-o output_file` :
 - Specify the output filename; not necessary when compiling to object code. If FILE is not specified, the default name is `a.out`
- `-C` :
 - Compile or assemble the source files, but do not link
- `-g` :
 - Include standard debugging information in the binary
- `-E` :
 - Stop after the preprocessing stage; do not run the compiler proper
- `-O` :
 - The bare `-O` option tells gcc to reduce both code size and execution time. It is equivalent to `-O1`

```
root@miraculous-desktop:~/hello# ls
hello.c Makefile
root@miraculous-desktop:~/hello# gcc hello.c -o hello.out
root@miraculous-desktop:~/hello# ls
hello.c hello.out Makefile
root@miraculous-desktop:~/hello#
```

```
root@miraculous-desktop:~/hello# gcc -E hello.c -o hello.cpp
root@miraculous-desktop:~/hello# ls
hello.c hello.cpp hello.out Makefile
root@miraculous-desktop:~/hello#
```



GCC COMMAND-LINE OPTIONS

- Code optimization problem
 - Code optimization is an attempt to improve performance
 - The trade-off is lengthened compile times and increased memory usage during compilation
- -I :
 - Add the directory to the list of directories to be searched for header files.
 - Example : `gcc program.c -I/usr/include`
- -L :
 - Add directory to the list of directories to be searched for library files.
 - Example : `gcc program.c -L/usr/lib`



MAKEFILE

- Makefile
 - The GNU make utility automatically determines which pieces of a large program need to be recompiled, and issues the commands to recompile them.



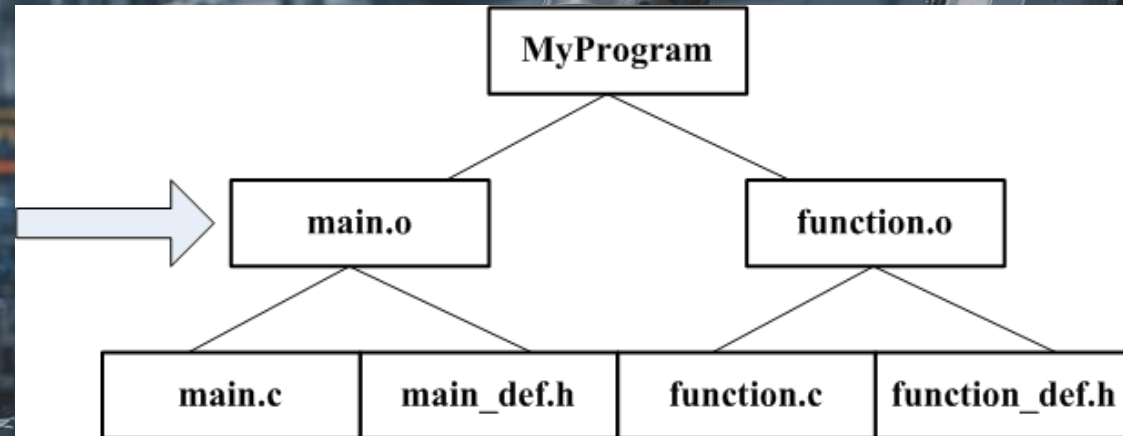
MAKEFILE

- A rule consists of the following
 - Target : A target is usually the name of a file that is generated by a program; examples of targets are executable or object files. A target can also be the name of an action to carry out, such as 'clean'.
 - Dependency : A dependency is a file that is used as input to create the target. A target often depends on several files.
 - Command : A command is an action that make carries out.
- target : dependency dependency [...]
- command
- command
- [...]



A SIMPLE MAKEFILE

```
MyProgram:main.o function.o
gcc -o MyProgram main.o function.o
main.o:main.c main_def.h
gcc -c main.c
function.o:function.c function_def.h
gcc -c function.c
```



- This makefile has three rules
- The first target, MyProgram, is called the default target
- MyProgram has two dependencies, main.o and function.o; these two files must exist in order to build MyProgram.



MAKEFILE

- How make processes a Makefile :
 - By default, make starts with the first target.
 - Thus, when you give the command: make; make reads the “Makefile” in the current directory and begins by processing the first rule.
 - Make compares the timestamp on the target to the timestamp of the dependencies.
 - If one or more of the dependencies is newer than the target, make rebuilds the target, assuming that the newer dependency implies some code change that must be incorporated into the target.



USING VARIABLES IN MAKEFILE

- Variables :
 - To simplify editing and maintaining makefiles, make allows you to create and use variables.
 - Define variables using the general form :
 - `VARNAME = text [...]`
 - Example :
 - `OBJS = main.o function.o`
 - `HDRS = main.c main_def.h`
 - To obtain `VARNAME`'s value, enclose it in parentheses and prefix it with a \$:
 - `$(VARNAME)`
 - Example:
 - `MyProgram:$(OBJS)`



USING VARIABLES IN MAKEFILE

- `$@` : the filename of a rule's target
- `$<` : the name of the first dependency in a rule
- `^` : space-delimited list of all the dependencies in a rule
- `$?` : space-delimited list of all the dependencies in a rule that are newer than the target
- `$(@D)` : the directory part of a target filename, if the target is in a subdirectory
- `$(@F)` : the filename part of a target filename, if the target is in a subdirectory



USING VARIABLES IN MAKEFILE

Variable	Description
CC	Program for compiling C programs; default value = gcc
CPP	C Preprocessor program; default value = cpp
RM	Program to remove files; default value = "rm -f"
ARFLAGS	Flags for the archive-maintenance program; default = rv
ASFLAGS	Flags for the assembler program; no default
CFLAGS	Flags for the C compiler; no default
CPPFLAGS	Flags for the C preprocessor; no default
LDFLAGS	Flags for the linker (ld); no default
AR	Archive-maintenance programs; default value = ar
AS	Program to do assembly; default value = as

❖ Example:

- `CROSS_COMPILE = arm-unknown-linux-gnu-`
- `CC = $(CROSS_COMPILE)gcc /* arm-unknown-linux-gnu-gcc */`
- `LD = $(CROSS_COMPILE)ld /* arm-unknown-linux-gnu-ld */`



PATTERN RULES

- `%o : %c`
 - tells make to build any object file filename.o from a source file filename.c
- Example :
 - `%o: $(SDIR)%c`
 - `$(CC) -c $(CFLAGS) $(CPPFLAGS) $< -o $@`



A SIMPLE MAKEFILE EXAMPLE

- programA.c
- programB.c
- programC.c
- def.h

```
#include <stdio.h>

void programA()
{
    print("ProgramA\n");
}
```

```
#include <stdio.h>

void programB()
{
    print("ProgramB\n");
}
```

```
#include <stdio.h>

void programC()
{
    print("ProgramC\n");
}
```

```
void programA();
void programB();
void programC();
```



A SIMPLE MAKEFILE EXAMPLE

- Program.c

```
#include <stdio.h>
#include <def.h>

int main(void)
{
    programA();
    programB();
    programC();
    return 0;
}
```

- Makefile

```
FILE=Program
CFLAGS= -g -wall
CC=gcc
LIBS=-lm
INCLUDES=
OBJS=programA.o programB.o programC.o
SRCS=programA.c programB.c programC.c

all:$(FILE)
$(FILE): $(FILE).o $(OBJS)
        $(CC) $(CFLAGS) $(INCLUDES) -o $@ $(FILE).o $(OBJS) $(LIBS)

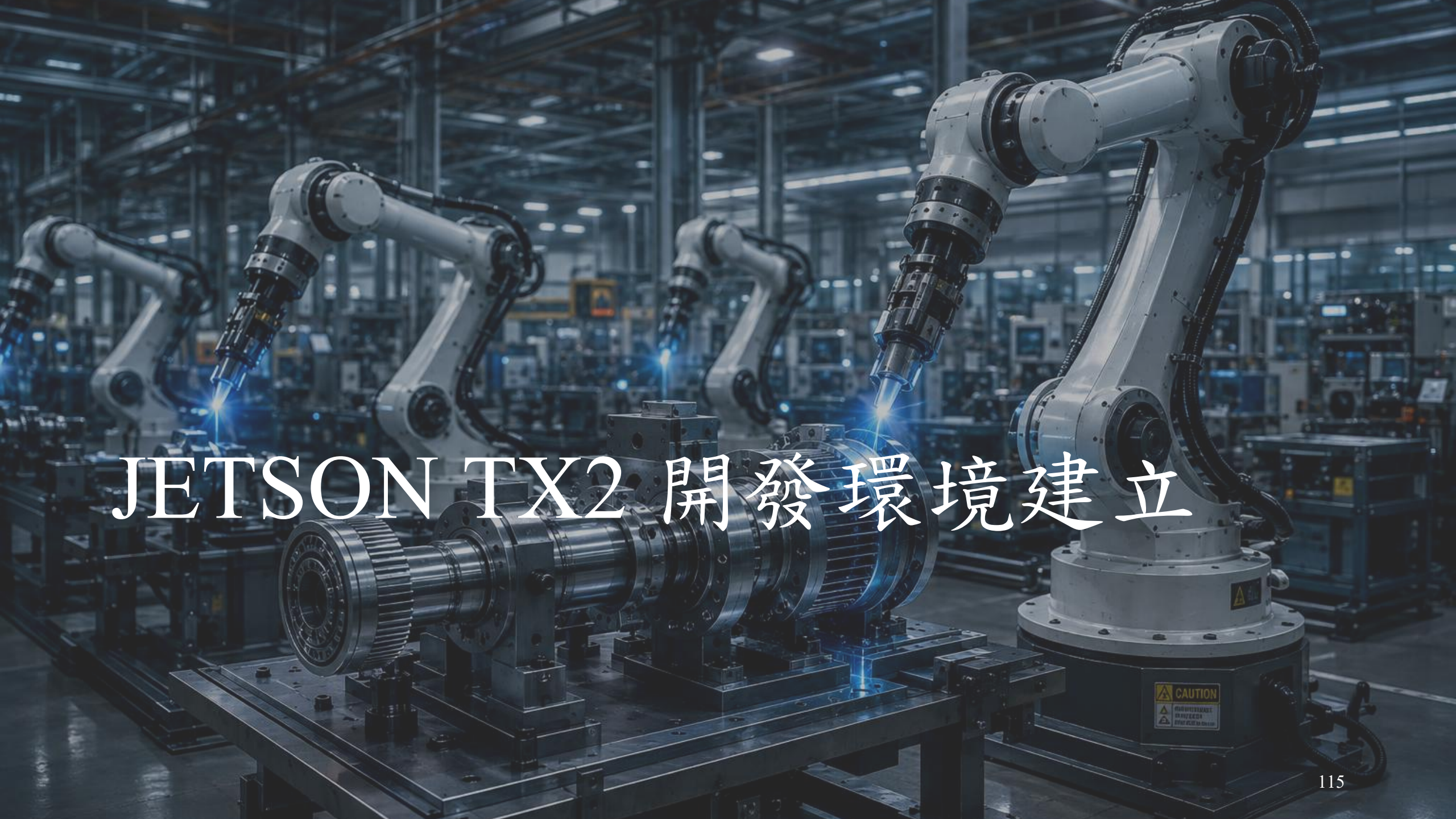
clean:
        rm *.o $(FILE)
```



A SIMPLE MAKEFILE EXAMPLE

- Type “make” to make this example

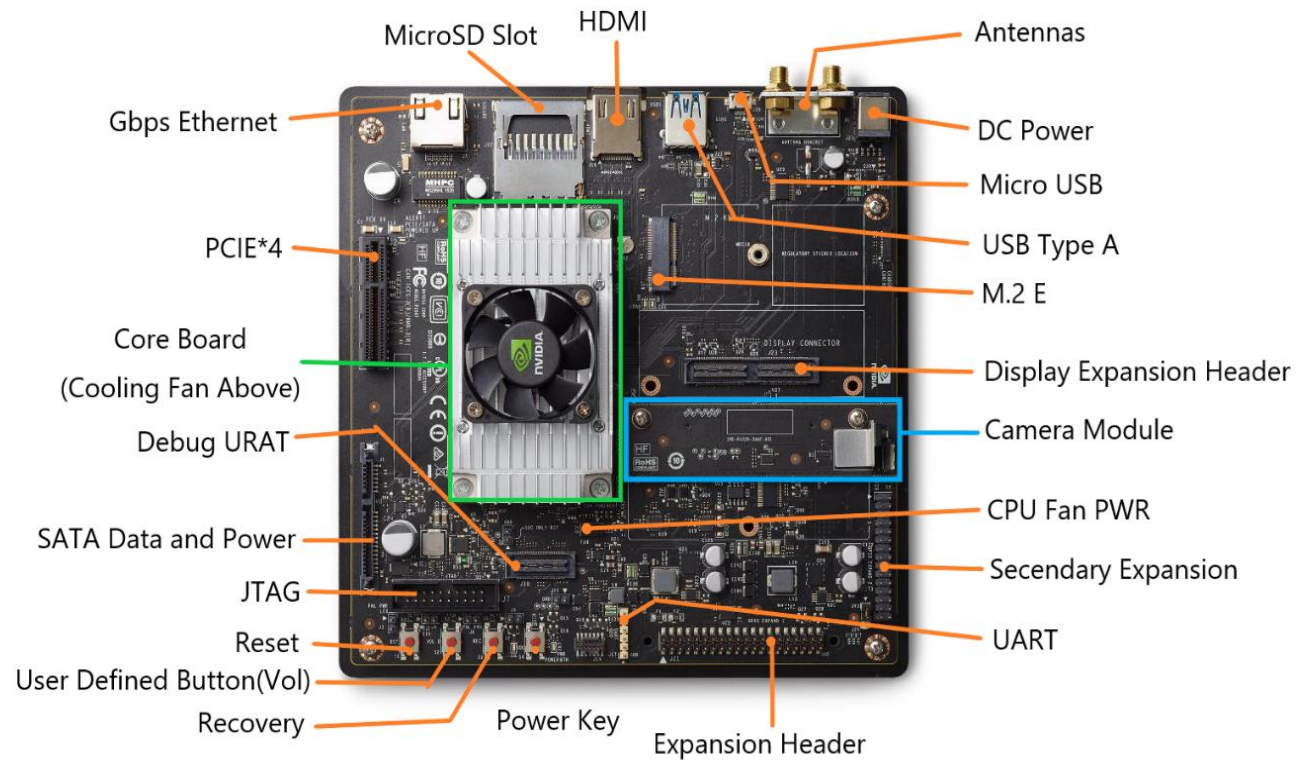
```
root@miraculous-desktop:~/Example# ls
def.h Makefile programA.c programB.c Program.c programC.c
root@miraculous-desktop:~/Example# make
gcc      -c -o Program.o Program.c
gcc      -c -o programA.o programA.c
gcc      -c -o programB.o programB.c
gcc      -c -o programC.o programC.c
gcc      -o Program Program.o programA.o programB.o programC.o -lm
root@miraculous-desktop:~/Example# ./Program
ProgramA
ProgramB
ProgramC
```

The background image shows a factory floor with several industrial robotic arms. The arms are white and black, with blue sparks emanating from their welding tips as they work on a large, complex metal component. The component is mounted on a heavy metal base. The scene is dimly lit, with the primary light source being the sparks from the welding process. The overall tone is industrial and technological.

JETSON TX2 開發環境建立



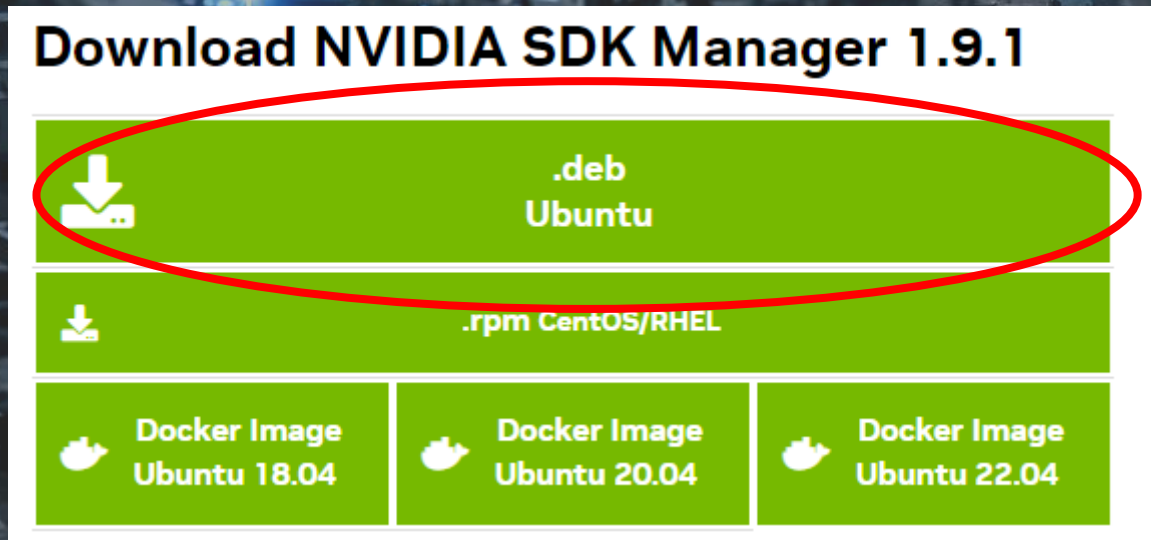
JETSON TX2硬體介紹



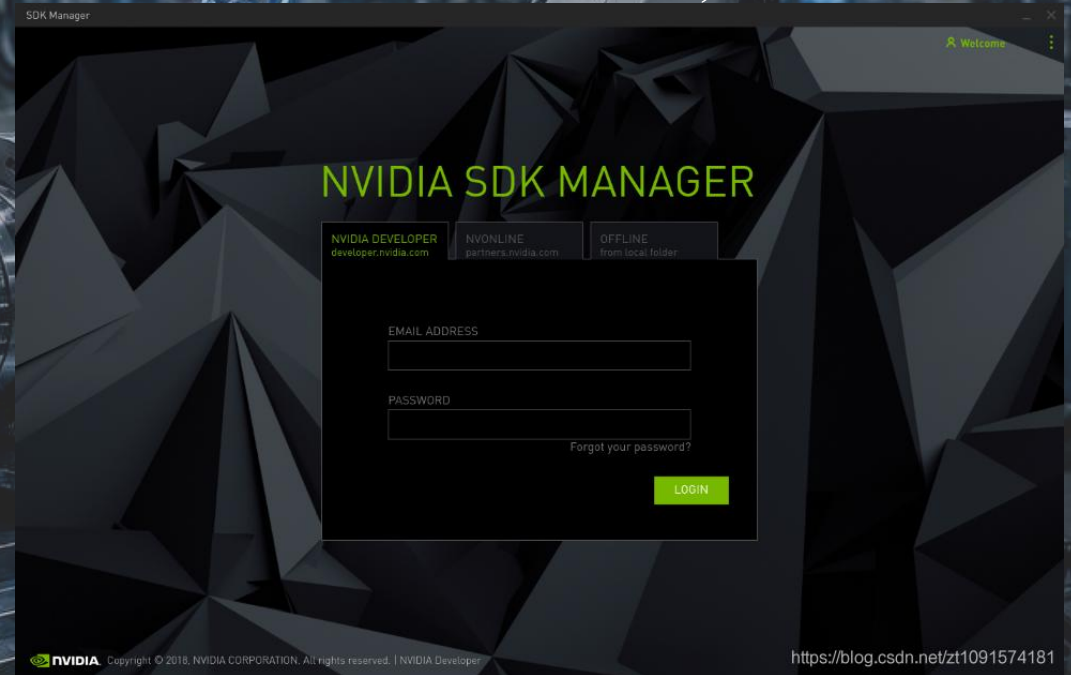


STEP 1 使用JETPACK燒錄TX2

- 先到 <https://developer.nvidia.com/drive/sdk-manager>
- Download SDK Manager
- 安裝在一台有Ubuntu的電腦(可用VMware架設的虛擬機)



點擊紅框內的.deb Ubuntu
下載 sdk-manager



安裝完成如此圖



STEP 2 TX2環境設定

• 選TX2

SDK Manager 0.9.12.4180

Hello Jim

STEP 01
DEVELOPMENT
ENVIRONMENT

STEP 02
DETAILS
AND LICENSE

STEP 03
SETUP
PROCESS

STEP 04
SUMMARY
FINALIZATION

Repair/ Uninstall

PRODUCT CATEGORY	Jetson	✓
HARDWARE CONFIGURATION	Host Machine ✓	Target Hardware
TARGET OPERATING SYSTEM	Linux JetPack 4	Jetson AGX Xavier (P2888)
		Jetson TX2 (P3310) Jetson TX2
		Jetson TX2i (P3489)
		Jetson Nano (Developer Kit version) (P3448)

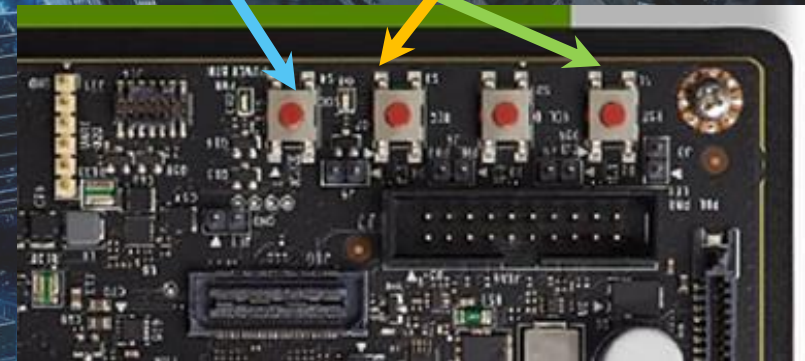
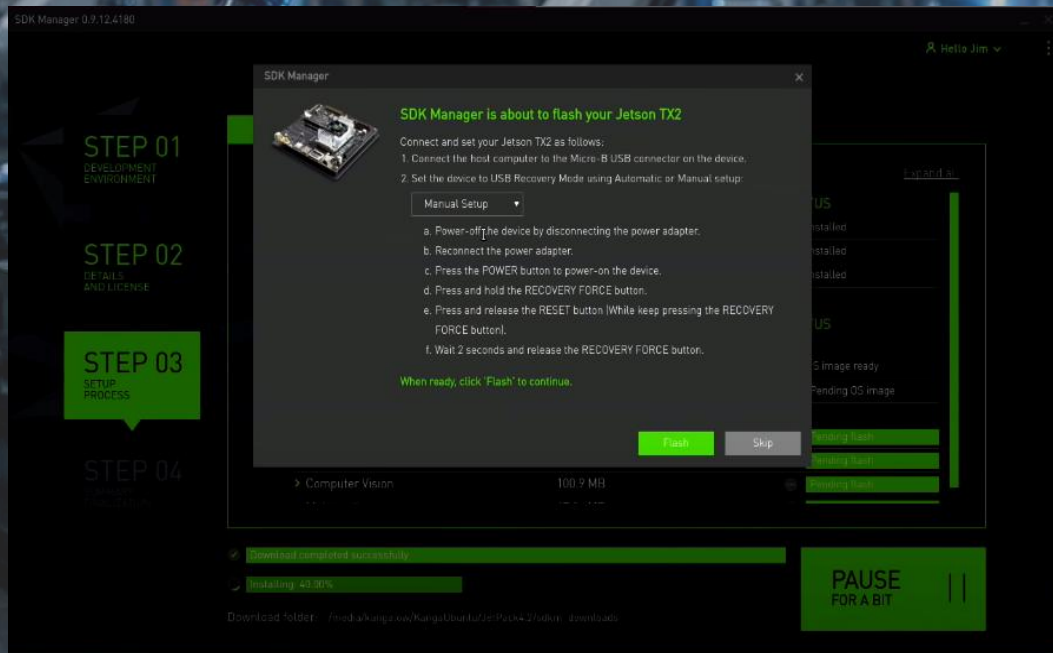
NVIDIA Copyright © 2019, NVIDIA CORPORATION. All rights reserved. | NVIDIA Developer



STEP 3 FLASH

• 選擇Manual Setup •

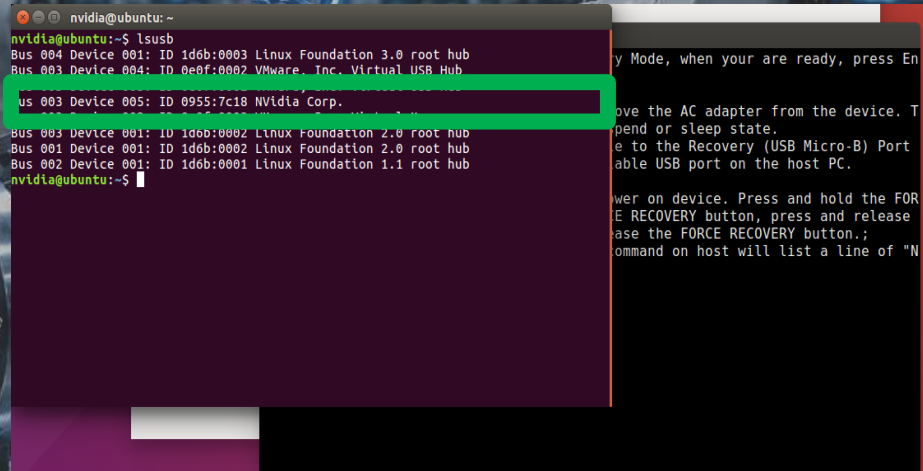
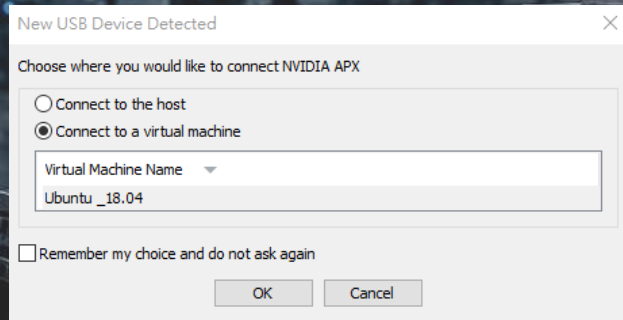
1. 關閉TX2
2. 接上Micro USB(連接TX2和PC)
3. 接上TX2電源
4. 按下TX2 **Power** 按鈕，**按住**旁邊的**Recovery**按鈕，並按一下**Reset**按鈕後放開所有按鈕。





STEP 3 FLASH

- 虛擬機的話就會出現下左圖，lsusb就會出現下右圖。
- 確認連接後，在上一頁彈出來的視窗點選"Flash"





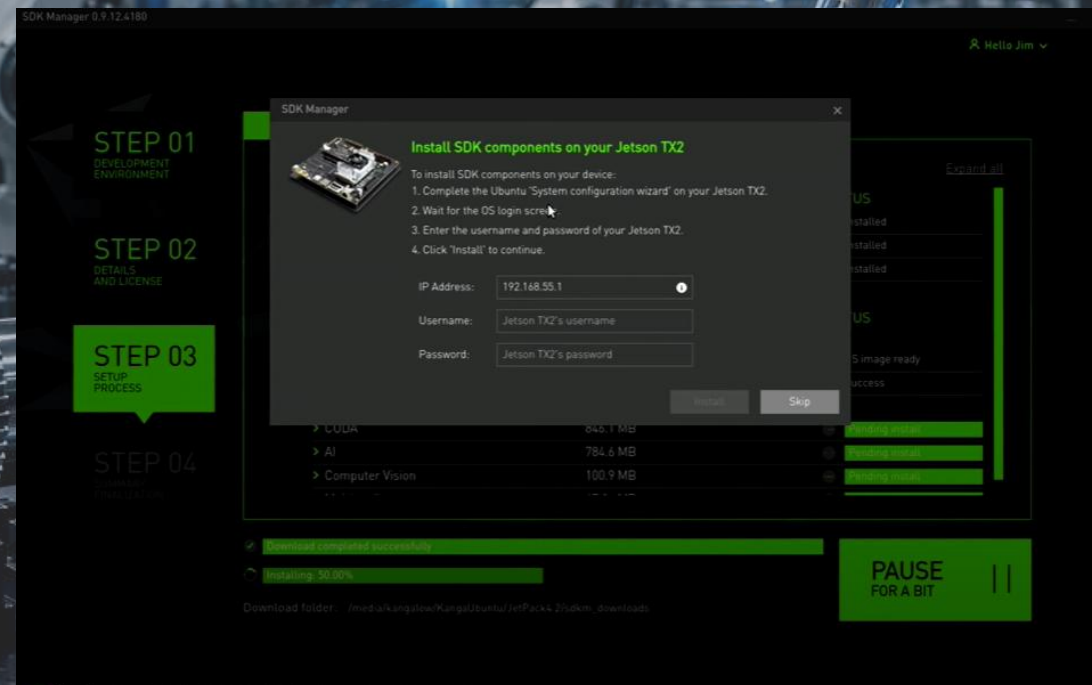
- 以下在TX2上進行
- (帳號密碼默認設定nvidia)
- 設定之後等待安裝

121



STEP 3 FLASH

- 回到電腦
- 填入名稱及密碼





安裝結束

SDK Manager 0.9.12.4180 Hello Jim

STEP 01
DEVELOPMENT
ENVIRONMENT

STEP 02
DETAILS
AND LICENSE

STEP 03
SETUP
PROCESS

STEP 04
SUMMARY
FINALIZATION

DETAILS

TERMINAL

JETPACK 4.2 LINUX FOR JETSON TX2 Expand all

HOST COMPONENTS	DOWNLOAD SIZE	STATUS
Computer Vision	130.1 MB	✓ Installed
Developer Tools	291.3 MB	✓ Installed

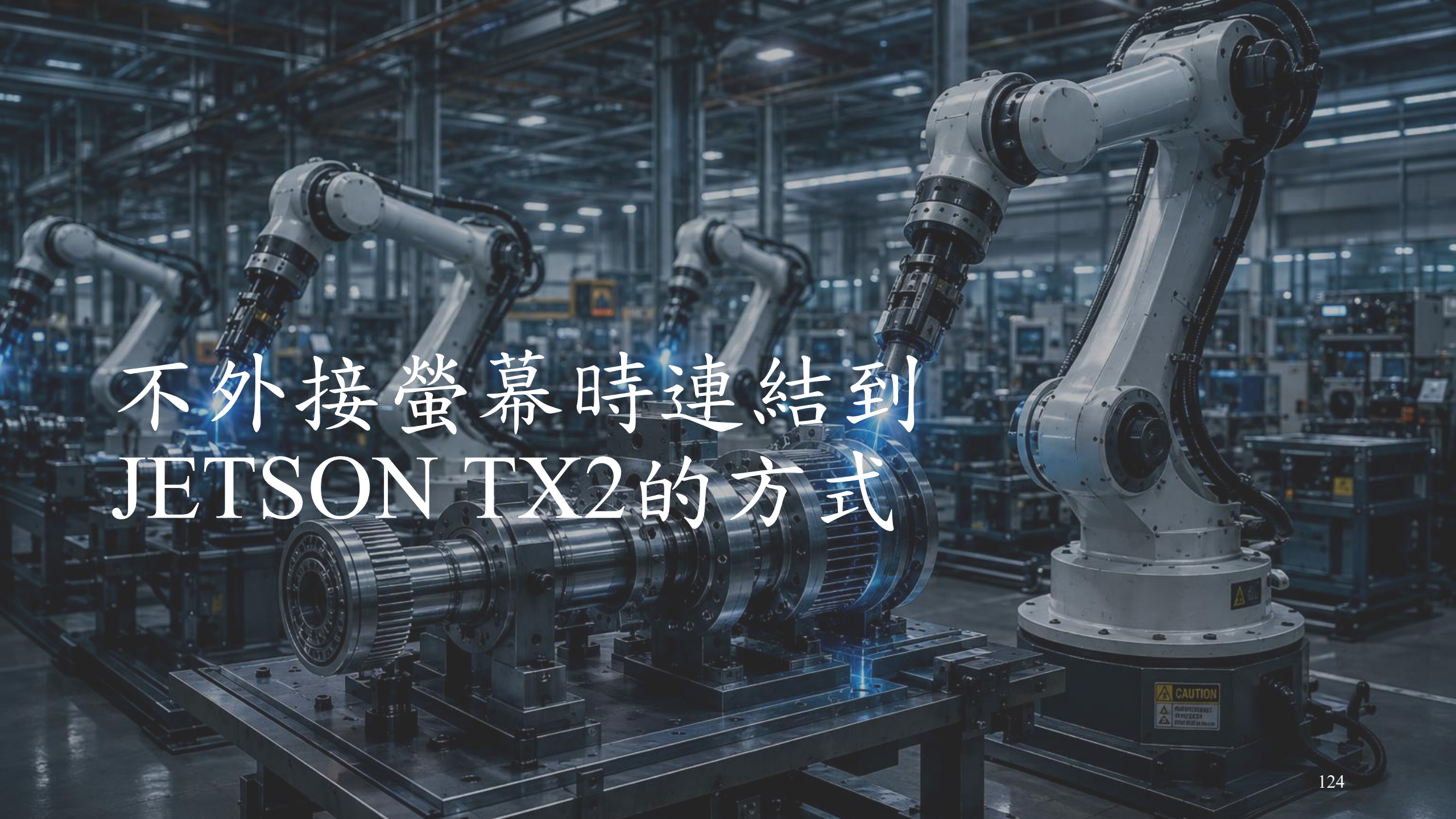
TARGET COMPONENTS	DOWNLOAD SIZE	STATUS
Jetson OS		
Jetson OS image	1,284 MB	✓ OS image ready
Flash Jetson OS		✓ Success
Jetson SDK components		
CUDA	846.1 MB	✓ Installed
AI	784.6 MB	✓ Installed
Computer Vision	100.9 MB	✓ Installed
Multimedia	67.1 MB	✓ Installed

✓ INSTALLATION COMPLETED SUCCESSFULLY. EXPORT LOGS

FINISH AND EXIT ✕

← BACK TO STEP 01

NVIDIA Copyright © 2019, NVIDIA CORPORATION. All rights reserved. | NVIDIA Developer

The background image shows a factory floor with several white industrial robotic arms. One arm in the foreground is positioned over a large, complex mechanical assembly that includes a large gear and various metal components. The scene is dimly lit with blue-toned lighting, creating a high-tech industrial atmosphere. The text is overlaid in the center-left area of the image.

不外接螢幕時連結到 JETSON TX2的方式



SSH是甚麼?與TELNET差別為何?

- Secure Shell (縮寫為SSH) 為一項建立在應用層和傳輸層基礎上的安全協定，為電腦上的Shell (殼層) 提供安全的傳輸和使用環境。
- 傳統的網路服務程式，如rsh、FTP、POP和Telnet其本質上都是不安全的；因為它們在網路上用明文傳送資料、用戶帳號和用戶口令，很容易受到中間人 (man-in-the-middle) 攻擊方式的攻擊。就是存在另一個人或者一台機器冒充真正的伺服器接收用戶傳給伺服器的資料，然後再冒充用戶把資料傳給真正的伺服器。
- 而SSH是目前較可靠，專為遠端登入對談和其他網路服務提供安全性的協定。利用SSH協定可以有效防止遠端管理過程中的資訊泄露問題。透過SSH可以對所有傳輸的資料進行加密，也能夠防止DNS欺騙和IP欺騙。
- SSH之另一項優點為其傳輸的資料可以是經過壓縮的，所以可以加快傳輸的速度。SSH有很多功能，它既可以代替Telnet，又可以為FTP、POP、甚至為PPP提供一個安全的「通道」。



在UBUNTU虛擬機中使用SSH連到TX2

- Step. 1
 - 開啟Ubuntu虛擬機中的終端機(快捷鍵：Ctrl + Alt + T)
- Step. 2
 - 輸入ssh -Y <使用者>@<TX2的IP>
 - 假設TX2的IP為192.168.137.15，使用者為nvidia(不要輸入ubuntu)
 - 輸入: ssh -Y nvidia@192.168.137.15
 - 其中 -Y 參數為支援XWindow顯示
- Step. 3
 - 第一此連線會詢問是否同意連結到此裝置，輸入yes
 - 接著輸入使用者的密碼就可以完成登入了



在UBUNTU虛擬機中使用SSH連到TX2

nvdi@tegra-ubuntu: ~

nvdi@ubuntu:~\$ ssh -Y nvdi@192.168.137.15

The authenticity of host '192.168.137.15 (192.168.137.15)' can't be established.
ECDSA key fingerprint is SHA256:oTgKigtYuJgE0GInUM9EkZ0hS6hfjvfr2ENHU7j+1wE.

Are you sure you want to continue connecting (yes/no)? yes

Warning: Permanently added '192.168.137.15' (ECDSA) to the list of known hosts.
Welcome to Ubuntu 16.04 LTS (GNU/Linux 4.4.38-tegra aarch64)

* Documentation: <https://help.ubuntu.com/>

718 packages can be updated.
462 updates are security updates.

New release '18.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

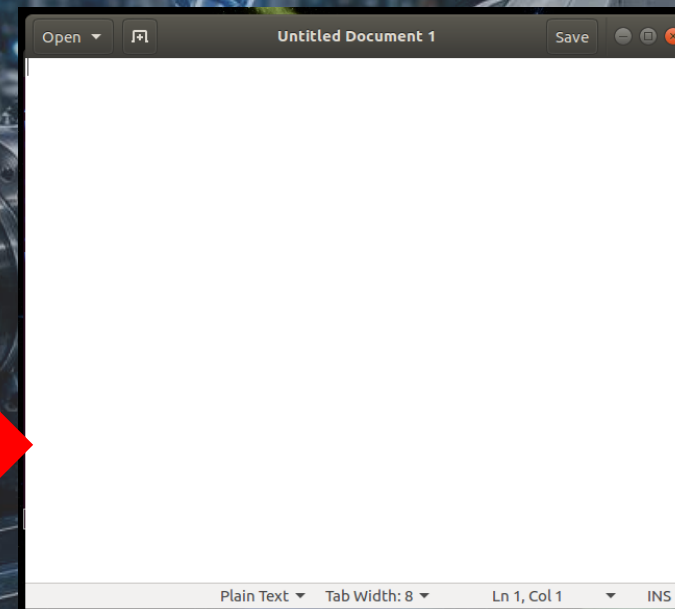
Last login: Thu Feb 13 09:17:19 2020



在UBUNTU虛擬機中使用SSH連到TX2

- 使用帶有參數-Y的ssh連入TX2可以映射XWindow到電腦上，我們可以輸入gedit來開啟圖型化視窗的文字編輯器看看是否成功

```
* Documentation: https://help.ubuntu.com/  
  
718 packages can be updated.  
462 updates are security updates.  
  
New release '18.04.3 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Thu Feb 13 09:17:19 2020  
nvidia@tegra-ubuntu:~$ gedit
```



A photograph of an industrial manufacturing environment. In the foreground, a large, complex metal component, possibly a motor or a large gear assembly, is mounted on a heavy metal base. Several robotic arms, painted in a light grey or white color, are positioned around the component. Each arm has a blue laser line projected from its end, indicating a precise welding or assembly process. The background shows a vast factory floor with various industrial equipment, including conveyor belts and other machinery, under bright overhead lighting. The overall scene conveys a sense of advanced automation and precision manufacturing.

傳送檔案到TX2嵌入式平台



傳送檔案到TX2

- 當我們需要將檔案或編譯過後的執行檔傳送到嵌入式平台TX2的時候，由於TX2內的Ubuntu預設有安裝ssh Server所以可以使用scp或sftp這兩種方式傳送檔案
 - scp (Secure copy):是指在本地主機與遠端主機或者兩台遠端主機之間基於Secure Shell (SSH) 協定安全地傳輸電腦檔案。適合單一的小檔案傳送
 - 複製本地端檔案到遠端主機
 - scp <本地端檔案路徑><使用者>@<遠端IP>:<目的端的路徑>
 - 從遠端主機複製檔案到本地端主機
 - scp <使用者>@<遠端IP>:<目的端的檔案路徑><本地端存的路徑位置>



傳送檔案到TX2

- 當我們需要將檔案或編譯過後的執行檔傳送到嵌入式平台TX2的時候，由於TX2內的Ubuntu預設有安裝ssh Server所以可以使用scp或sftp這兩種方式傳送檔案
 - sftp (Secret File Transfer Protocol): 是一資料流連線，提供檔案存取、傳輸和管理功能的網路傳輸協定。
- * 在上課期間我們傳送檔案到TX2嵌入式平台，會比較推薦使用sftp的方式傳輸



如何使用SFTP傳送檔案到TX2

- 在我們開始使用sftp傳送檔案到TX2嵌入式平台之前，我們必須安裝FTP軟體，我們可以在Ubuntu上透過終端機來安裝filezilla這套FTP軟體
- 步驟 1：打開終端機，輸入指令 `sudo apt-get install filezilla`

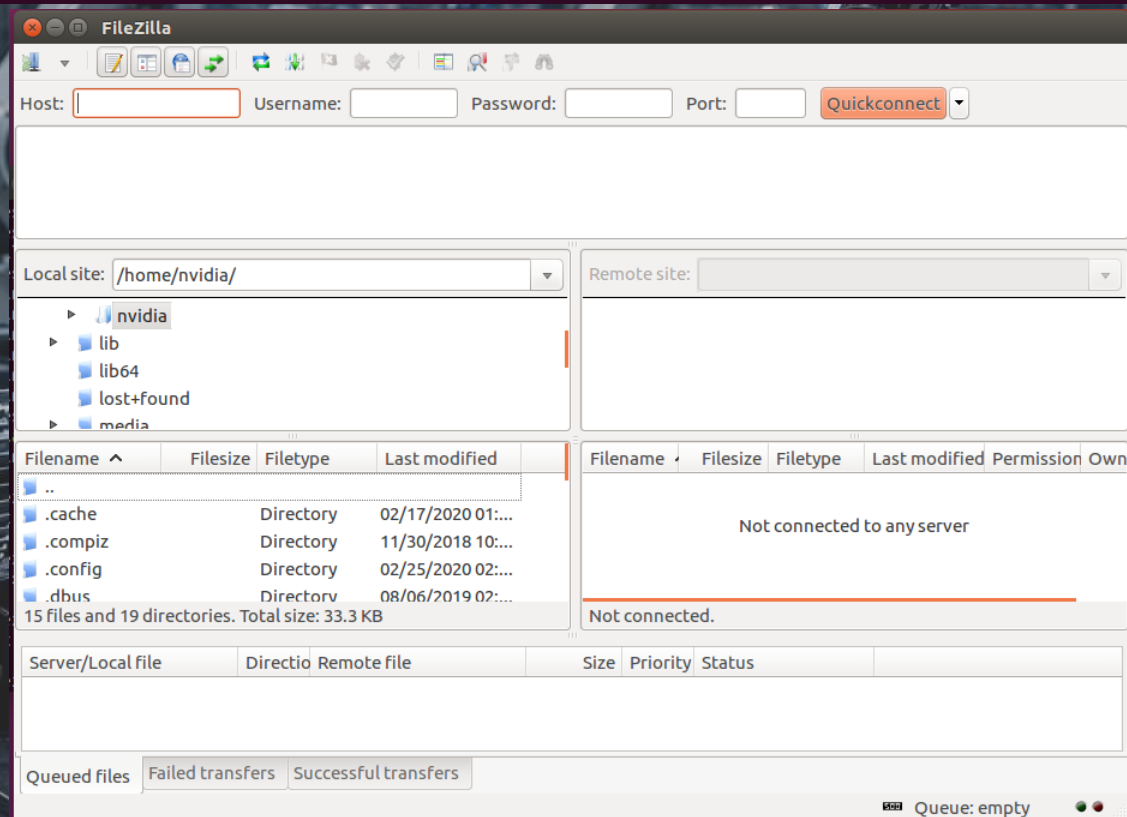
```
nvidia@ubuntu:~$ sudo apt-get install filezilla
[sudo] password for nvidia:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-headers-4.15.0-29 linux-headers-4.15.0-29-generic
  linux-image-4.15.0-29-generic linux-modules-4.15.0-29-generic
  linux-modules-extra-4.15.0-29-generic
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  filezilla-common libfilezilla0 libpugixml1v5 libwxbase3.0-0v5
  libwxgtk3.0-0v5
The following NEW packages will be installed:
  filezilla filezilla-common libfilezilla0 libpugixml1v5 libwxbase3.0-0v5
  libwxgtk3.0-0v5
0 upgraded, 6 newly installed, 0 to remove and 103 not upgraded.
Need to get 8,977 kB of archives.
After this operation, 36.9 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```



如何使用SFTP傳送檔案到TX2

- 步驟2：終端機輸入 filezilla，測試是否有安裝成功。

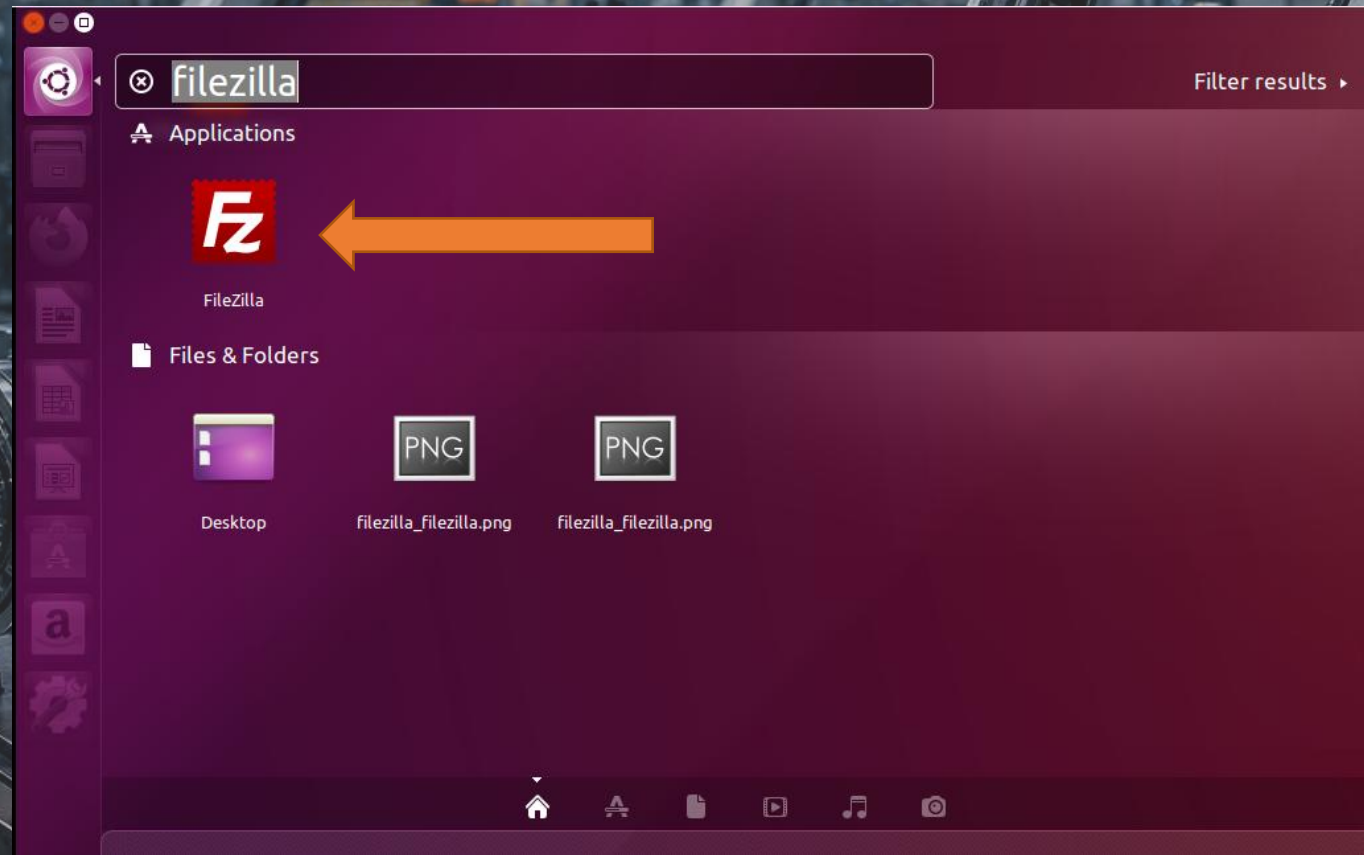
```
nvidia@ubuntu:~$ filezilla  
Reading locale option from /home/nvidia/.config/filezilla/filezilla.xml
```





如何使用SFTP傳送檔案到TX2

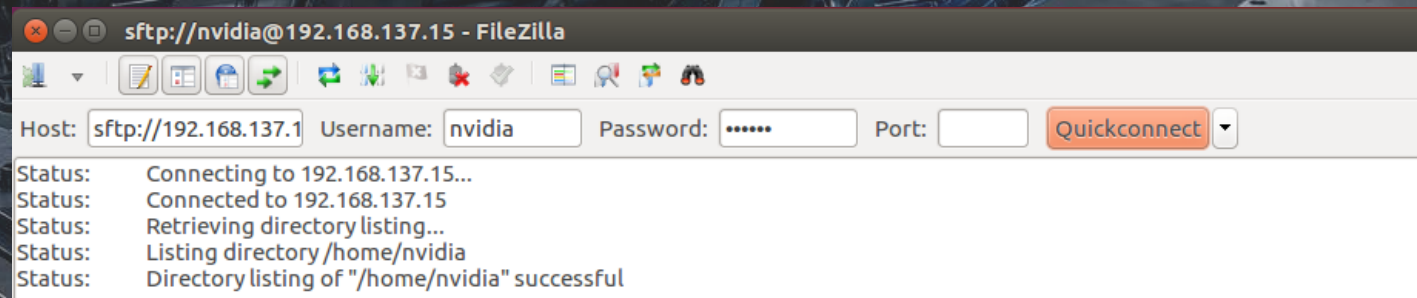
- 步驟3：安裝好後點選側邊欄最上的圖示搜尋FileZilla，請點選並執行此軟體





如何使用SFTP傳送檔案到TX2

- 步驟4：開啟 FileZilla後在上方有數個欄位
 - Host: 輸入sftp://<TX2的IP>
 - 例如IP為192.168.137.15 則輸入 sftp://192.168.137.15
 - Username: 輸入使用者名稱(預設為nvidia)
 - Password: 輸入使用者密碼(預設為nvidia)
 - Port: 我們沒有更改port號，所以此欄位空白不用輸入
 - 輸入完成後按下右方的Quickconnect後就可以快速連到TX2嵌入式平台了
 - *若連結時有出現Unknown host key的訊息請直接按下OK





如何使用SFTP傳送檔案到TX2

- 連線完成後可以使用簡單的拖拉放方式傳送檔案到遠端有寫入權限的目錄下

